Round-Trip Refinement: A Training-Free Sampler for Next-Concept Language Models

Pierre Khoury
MSc Advanced Computing, King's College London
pierre.khoury@kcl.ac.uk

Abstract

Next-Concept Modeling (NCM) generates text coarse to fine: a multiscale residual VQ-VAE turns a sequence into twelve levels of discrete codes, and a transformer predicts one level per pass. We find that the code stacks NCM samples are not the stacks its own tokenizer assigns to the text they decode to: re-encoding a sample rewrites 61–96% of the codes at each of the six coarsest levels, yet decodes back to the same text for 98% of samples. We propose *round-trip refinement*, a training-free sampler: decode the draft, re-encode the text, keep the re-encoded coarse levels, and regenerate the fine levels with the unchanged generator. On the released TinyStories and LM1B checkpoints, with the authors' own sampling and evaluation code, one round lowers GPT-2 Large generative perplexity by 17% and 22% at equal unigram entropy for six extra passes; two or three rounds reach −31% and −29%. Resampling the fine levels without re-encoding gives nothing, and re-quantizing the latent instead of the text recovers at most half. The gain is not temperature in disguise: at matched corpus diversity refinement sits 10–15% below the temperature frontier, exact 11-pass twins without the round trip gain nothing, and it does not sharpen real human text. On TinyStories, how non-canonical a draft is predicts its quality (Spearman 0.37) and the round trip helps most where the rewrite is largest, while selecting drafts by canonicality only trades diversity for perplexity. Paired with a per-level temperature schedule (schedule plus round trip), the samplers are about 21% more fluent than the released one at equal or higher diversity. Everything replicates on held-out seeds, three model sizes and a second scorer; an LLM judge prefers refined stories (0.63) and MAUVE rises from 0.88 to 0.94–0.96. Refinement gives small gains on hard Sudoku and XSum summaries, none on WMT14 translation or on the VAR image model, whose tokenizer round trips are lossy; seven training-side ideas, including distilling the refined sampler back into the generator, did not beat it at matched diversity. Code: https://github.com/PierreKhoury1/ncm-round-trip.

1 Introduction

Diffusion language models generate in parallel but need many denoising steps, each over the full sequence [28, 31]. Next-Concept Modeling [NCM; 13] takes a different route, borrowed from next-scale prediction in vision [12, 30]: a multiscale residual VQ-VAE encodes a 128-token sequence into K=12 levels of codes, from one code for the whole sequence to 128 codes at the finest level, and a transformer generates the levels coarse to fine, all codes of a level in parallel. NCM reaches the generative perplexity of strong diffusion baselines with an order of magnitude fewer function evaluations (NFE).

Each NCM level is sampled conditioned on the generator's own samples of the coarser levels. In training, by contrast, the coarser levels are the tokenizer's codes for real text, perturbed by uniform code replacement ["conditioning corruption"; 13, §3.4]. Residual quantization is many-to-one: many code stacks decode to the same tokens, and the tokenizer always produces one particular stack, the greedy residual quantization of its encoder output. A sampled stack need not be that one. We measure this directly: decoding a generated sample and re-encoding the text changes 61–96% of the codes at each of the six coarsest levels (20–72% at the finer ones), yet decoding the re-encoded stack gives back exactly the same text for 98% of samples (Section 5.4).

This suggests a simple repair. Once a draft exists, the tokenizer can tell us how *it* would have encoded that text. We keep the re-encoded coarse levels, which now form a stack of the kind the generator saw in training, and let the generator redo the fine levels on top of them (Figure 1). The procedure needs no training, no new parameters and no change to the authors' sampler or decoder; it reuses the released generator, tokenizer and evaluation scripts as they are.

Contributions.

- A diagnosis: NCM's sampled code stacks are non-canonical (not the stacks the tokenizer assigns to their own decoded text), and re-encoding replaces them by canonical ones without changing the text (Section 5.4). On TinyStories, how non-canonical a draft is predicts how bad it is (Spearman 0.37 with its Gen. PPL), and the round trip helps most where the rewrite is largest.
- Round-trip refinement, a training-free sampler with one knob, the restart level k (Section 3), and ablations showing that the re-encoding through text, not the extra passes, carries the gain; plus a free per-sample gate on the same signal that spends rounds only where the draft drifts (Section 5.10).
- An evaluation on nine released checkpoints under the authors' protocols, with the temperature frontier, three diversity measures and exact 11-pass temperature twins as controls (Section 5): −17% to −31% gen-PPL on TinyStories and −22% to −29% on LM1B large, 10–15% beyond the temperature frontier, and clear failure cases (translation with a lossy tokenizer, easy Sudoku).
- Evidence beyond Gen. PPL: held-out seeds, an LLM judge, MAUVE against human text, a trend over three model sizes, and a reasoning task (Sudoku) where refinement fixes puzzles without breaking solved ones (Sections 5.6–5.8).
- Negative results for seven training-side ideas, including the two the NCM paper lists as future work and two pilots at the paper's batch size, so that others need not repeat them (Section 6).

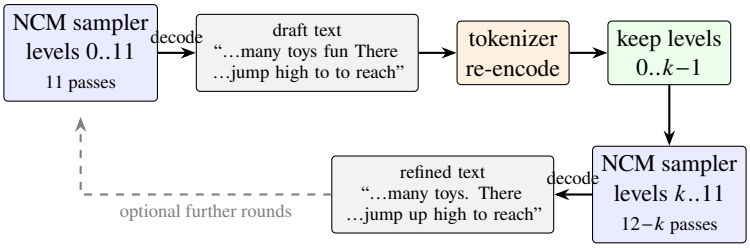


Figure 1: Round-trip refinement (RT_k). The draft is decoded to text and re-encoded by the tokenizer; the re-encoded coarse levels replace the sampled ones and the generator regenerates levels $k..11$. The text is a real example (TinyStories, seed 1, sample 4; full text in Appendix B).

2 Background: Next-Concept Modeling

Tokenizer. An encoder E maps a token sequence $x \in \mathcal{V}^L$ (L=128) to features $z \in \mathbb{R}^{L\times d}$. A multiscale residual quantizer with level lengths $\ell_0, \ldots, \ell_{K-1}$ (for TinyStories $1, 8, 16, 24, 32, 40, 52, 64, 76, 92, 108, 128$) then picks codes greedily,

$$r_k = \arg\min_j \left\| \mathrm{Down}_{\ell_k}(z - \hat{z}_{k-1}) - e_{k,j} \right\|, \qquad \hat{z}_k = \hat{z}_{k-1} + \phi_k\big(\mathrm{Up}_L(e_{k,r_k})\big), \tag{1}$$

with codebooks e_k of 4,096 entries and $\hat{z}_{-1}$=0. Read left to right: $z - \hat{z}_{k-1}$ is the *residual*, the part of the features that the coarser levels $0..k-1$ have not yet explained; Down_{ℓ_k} shrinks it from L positions to the ℓ_k positions of level k; the arg min picks, at each of those positions, the nearest entry $e_{k,j}$ of level k's codebook, and its index $r_k \in \{0, \ldots, 4095\}^{\ell_k}$ is the level-k code; Up_L stretches the chosen entries back to L positions, a per-level projection ϕ_k maps them into feature space, and the result is added to the running reconstruction $\hat{z}_k$. Level 0 has one position (ℓ_0=1), so r_0 is a single code for the whole sequence; level 11 has ℓ_{11}=L=128, one code per token. A decoder D maps the final reconstruction $\hat{z}_{K-1}$ back to token logits. We write $Q(E(x))$ for the full code stack of x and $\mathrm{dec}(r)$ for decoding a stack to tokens with the authors' token picker (argmax, with frequency and adjacency penalties on LM1B).

Generator. The NCM transformer predicts all ℓ_k codes of level k in one pass from the reconstruction $\hat{z}_{k-1}$ resized to ℓ_k positions, with block-diagonal attention so that pass k processes only ℓ_k positions. Unconditional generation draws r_0 from a unigram prior and then runs $K-1$=11 passes. Training uses teacher forcing on $Q(E(x))$ for real text, with a share of the input codes replaced uniformly at random (mean rate 0.5, decreasing with level), while targets stay clean.

Algorithm 1 Round-trip refinement with restart levels $k_1, \ldots, k_m$

$r \leftarrow$ NCM-SAMPLE() ▷ draft: r_0 from the prior, then $K-1$ passes
$x \leftarrow \mathrm{dec}(r)$
for $k \in (k_1, \ldots, k_m)$ **do**
  $r' \leftarrow Q(E(x))$ ▷ re-encode the current text
  $r \leftarrow$ NCM-SAMPLE(levels $0..k-1$ fixed to $r'_{0..k-1}$) ▷ $K-k$ passes
  $x \leftarrow \mathrm{dec}(r)$
end for
return x

3 Round-Trip Refinement

Algorithm 1 gives the sampler. One round with restart level k, written RT_k, costs $K-k$ extra generator passes plus one tokenizer encoder pass. Multiple rounds are written $\mathrm{RT}_5{\times}2$ (two rounds from level 5) or $\mathrm{RT}_3{+}\mathrm{RT}_6{+}\mathrm{RT}_9$ (three rounds from successively finer levels). Because pass k processes ℓ_k positions, regenerating levels 6..11 reprocesses 520 of the 641 positions of a full sample; FLOPs therefore grow by about $1.8\times$ for RT_6 and $3.3\times$ for three rounds, somewhat more than the NFE ratios of $1.5\times$ and $2.6\times$ suggest; measured wall-clock follows the FLOPs ($1.9\times$ and $3.8\times$ on one GPU, Section 5.11).

Implementation. We do not reimplement sampling. The authors' `generate` calls a per-level sampling function once per level; we wrap that function so that, for levels below k, it returns the re-encoded codes instead of sampling (Appendix A). Code embeddings, level encodings, heads, attention masks and the token picker are therefore exactly the released ones. As a check, forcing *every* level to the re-encoded stack reproduces the draft text exactly on TinyStories (8/8 samples tested), 27/32 on XSum and only 15/32 on WMT14, whose tokenizer round trip is lossy (Section 5.3).

Ablations. Two controls isolate where the gain comes from. *Resample*$_k$ keeps the draft's own codes for levels below k and regenerates the rest; it costs the same as RT_k but involves no re-encoding, so it draws the fine levels from the same conditional distribution as the draft did. *LatentRQ*$_k$ re-quantizes the draft's latent $\hat{z}_{K-1}$ with Eq. (1) instead of decoding and re-encoding the text; it is the text analogue of scale-travel in DiverseVAR [21].

4 Experimental setup

Checkpoints and sampling. We use the released NCM checkpoints for TinyStories [7] (88.6M generator), LM1B [5] small, medium and large, XSum [20] and WMT14 de–en [2], with the released evaluation configurations: temperature 1, no top-k/top-p and no guidance for TinyStories and LM1B (LM1B with the configured frequency and adjacency penalties in the token picker), greedy decoding for XSum and WMT14. Refinement rounds use the same settings as the draft unless stated otherwise.

Metrics. For TinyStories we report generative perplexity (Gen. PPL) under GPT-2 Large [27] and the mean per-sample unigram entropy H of GPT-2 tokens, as in the NCM paper; for LM1B we use the authors' FLM protocol (corpus perplexity over the first 128 GPT-2 tokens of the decoded ids, entropy over raw ids). Because Gen. PPL can be lowered by reducing diversity [8, 24], we add corpus distinct-3 (unique word trigrams over all trigrams), cross-sample 4-gram overlap (xs-4, share of a sample's 4-grams that occur in another sample; lower is more diverse), and the rate of adjacent doubled words, and we compare every sampler with the *temperature frontier*: the released sampler at $\tau \in \{1, 0.95, 0.9, 0.85, 0.8, 0.7\}$, interpolated at the same distinct-3 (TinyStories) or entropy (LM1B). XSum is scored with ROUGE-1/2/L [17] plus GPT-2 Gen. PPL of the summaries, WMT14 with sacreBLEU [23] and chrF. Unconditional results average three seeds of 256 samples each. All samplers of a seed share their level-0 draws; drafts are identical across samplers only for the first batch of each seed (samples 0–127 on TinyStories, 0–31 on LM1B), because refinement rounds consume random numbers before the second batch. Conditional results use the first 1,000 test documents.

Reproduction. Our pipeline gives 16.15 ± 0.17 Gen. PPL for the released TinyStories sampler (seeds 1–3) and 16.32 on seeds 4–6; the closest number in the NCM paper is 15.41, from its block-diagonal attention ablation, which is not the released evaluation run. All comparisons below are against our own reproduction, under identical code and seeds.

5 Results

5.1 TinyStories

Sampler	NFE	Gen. PPL ↓	Δ	H	dist-3 ↑	xs-4 ↓	dup% ↓	vs. τ frontier
released (τ=1)	11	16.15±0.17	–	3.980	0.749	0.162	0.39	–
τ=0.95	11	15.13±0.27	-6.3%	3.980	0.741	0.173	0.36	–
τ=0.9	11	14.64±0.15	-9.4%	3.979	0.735	0.179	0.36	–
τ=0.85	11	13.75±0.41	-14.9%	3.968	0.723	0.188	0.32	–
τ=0.8	11	13.34±0.25	-17.4%	3.968	0.719	0.191	0.35	–
τ=0.7	11	12.59±0.18	-22.1%	3.962	0.699	0.213	0.36	–
RT_9	14	15.13±0.15	-6.4%	3.980	0.747	0.165	0.33	-4.0%
RT_8	15	14.76±0.31	-8.6%	3.980	0.744	0.168	0.28	-6.0%
RT_7	16	13.84±0.33	-14.3%	3.974	0.740	0.171	0.24	-8.3%
RT_6	17	13.36±0.12	-17.3%	3.974	0.740	0.170	0.23	-12.4%
RT_5	18	12.90±0.47	-20.1%	3.975	0.731	0.181	0.31	-10.9%
RT_4	19	12.90±0.31	-20.2%	3.976	0.727	0.184	0.33	-8.8%
RT_3	20	13.24±0.15	-18.0%	3.969	0.720	0.195	0.39	-2.2%
RT_6×2	23	12.64±0.34	-21.8%	3.973	0.730	0.178	0.19	-12.3%
RT_4+RT_8	23	12.34±0.18	-23.6%	3.969	0.724	0.187	0.24	-10.1%
RT_5×2	25	11.70±0.31	-27.6%	3.970	0.718	0.191	0.25	-12.9%
RT_3+RT_6+RT_9	29	11.17±0.24	-30.9%	3.963	0.708	0.205	0.22	-13.7%
RT_6, $\tau_{\geq 6}$=0.8	17	12.96±0.17	-19.8%	3.969	0.739	0.171	0.23	-14.4%
RT_5, $\tau_{\geq 6}$=0.8	18	12.48±0.44	-22.7%	3.974	0.729	0.182	0.27	-13.1%
RT_5×2, $\tau_{\geq 6}$=0.9	25	11.56±0.16	-28.4%	3.969	0.717	0.193	0.27	-13.1%
human text (validation)	–	8.51	–	–	0.775	0.139	<0.01	–

Table 1: TinyStories, released checkpoint, GPT-2 Large, 3 seeds × 256 samples (mean ± s.d. over seeds). Δ: change in Gen. PPL against the released sampler. Last column: Gen. PPL relative to the temperature curve (τ=0.7..1.2) interpolated at the same corpus distinct-3, per seed then averaged (negative = better than changing the temperature). Every refinement variant beats the released sampler on all three seeds.

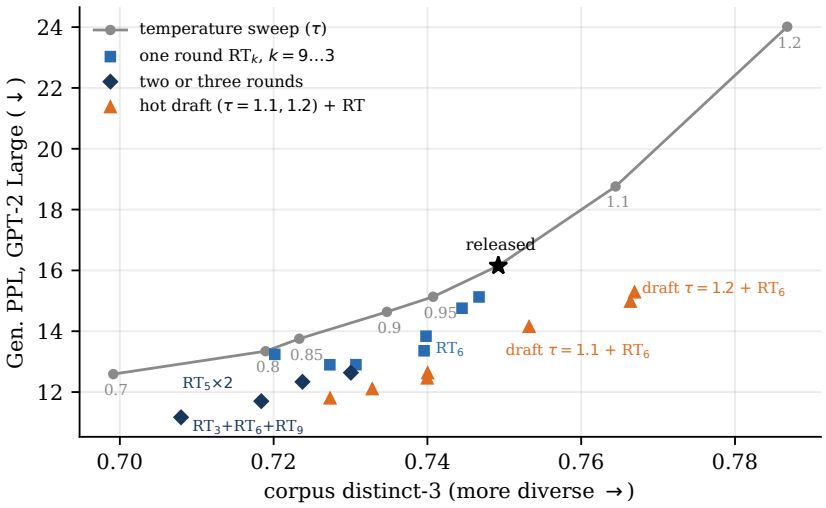


Figure 2: TinyStories: Gen. PPL against corpus distinct-3. Lowering the temperature buys fluency with diversity along the grey curve; round-trip refinement lands below it at every restart level, and further rounds move along a lower curve.

Main result. Table 1 and Figure 2 give the main result. The released sampler scores 16.15 Gen. PPL and human-written stories score 8.51, so the released model sits 7.6 points above human text. One round from level 6 (RT_6)

lowers Gen. PPL to 13.36, a 17.3% drop that holds on every seed, while unigram entropy stays at 3.974 against 3.980: the stories read more fluently to GPT-2 Large, and the words they use are spread as widely as before. The restart level is the only knob. Restarting from a coarser level regenerates more of the story, and this helps up to k=4–5, where the gain reaches 20%; restarting from level 3 rewrites too much, the draft's content starts to drift, and the gain falls back to 18% while diversity drops (distinct-3 0.720 against 0.740 for RT_6). Further rounds keep improving: $RT_5{\times}2$ reaches 11.70 (−27.6%) and $RT_3{+}RT_6{+}RT_9$ reaches 11.17 (−30.9%). The latter closes 65% of the 7.6-point gap between the released sampler and human text, at the price of 29 generator passes instead of 11.

Not a temperature effect. Lowering the temperature also lowers Gen. PPL: τ=0.8 gives 13.34, the same number as RT_6, so perplexity alone cannot tell the two apart. The diversity columns can. Temperature reaches 13.34 by making the stories more alike: distinct-3 falls from 0.749 to 0.719, and the share of a story's 4-grams that also occur in another story (xs-4) rises from 0.162 to 0.191. RT_6 keeps distinct-3 at 0.740 and xs-4 at 0.170, close to the released sampler. To compare the two fairly we interpolate the temperature curve (τ from 0.7 to 1.2) at each sampler's own distinct-3 and ask how much lower its perplexity is than temperature would give at that diversity (last column of Table 1). By that measure single rounds are 6–12% better than temperature for k=5..8 and multiple rounds 10–14%; sampling levels 6–11 at τ=0.8 in both the draft and the round ($\tau_{\geq 6}$=0.8) gives the largest single-round margin, 14.4%. Refinement also removes a kind of error that temperature does not touch. Adjacent doubled words ("to to", "quickly quickly") are a typical failure of predicting a whole level in parallel, since neighbouring positions cannot see each other's choice; they fall from 0.39% of words to 0.23% under RT_6, while every temperature setting leaves them at 0.32–0.36%. The re-encoded coarse levels therefore change *what* the fine levels produce, rather than only sharpening the same distribution.

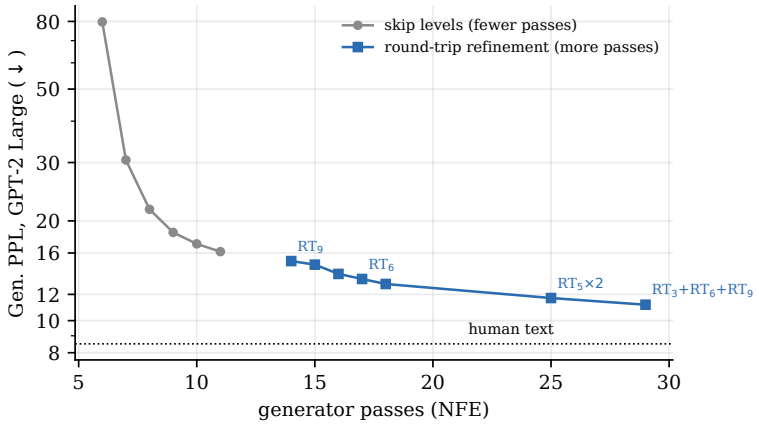


Figure 3: Quality against generator passes on TinyStories (GPT-2 Large). Grey: the released generator with middle levels skipped (fewer passes), which degrades quickly. Blue: round-trip refinement, which turns extra passes into steady gains. Dotted: human-written validation openings.

Compute. Figure 3 places refinement on a quality–compute curve, with the number of generator passes on the horizontal axis. Going left, i.e. saving passes by skipping middle levels of the released sampler, is expensive: one skipped pass already costs +6% Gen. PPL and four skipped passes +89%, so the released 11-pass budget is close to the minimum the model needs. Going right, refinement turns 3–18 extra passes into −6% to −31%, a roughly linear return that no setting of the released sampler offers. NCM's speed advantage over diffusion LMs is 10–31× in wall-clock [13], so even the three-round sampler (29 passes, about 3.3× the FLOPs and 3.8× the wall-clock) stays far cheaper than the 128–1,024-step diffusion samplers NCM is compared with.

5.2 LM1B

A second domain and three model sizes. The result carries over to LM1B, which differs from TinyStories in domain (news sentences rather than children's stories), tokenizer (BERT word pieces) and evaluation protocol (the authors' FLM scoring; Table 2, Figure 4). Perplexities are higher here because news sentences are harder to predict than children's stories, so the relative changes are what to read. On the small model one round lowers Gen. PPL by 11–18% depending on the restart level, on the large model by 6–22%, on every seed and at equal or slightly *higher* entropy. Temperature cannot do this: τ=0.8 reaches −16% on the large model only by dropping entropy from

Sampler	NFE	Gen. PPL ↓	Δ	H	dist-3 ↑	xs-4 ↓	seeds better
LM1B small							
released (τ=1)	11	94.79±2.56	–	4.313	0.864	0.055	–
τ=0.9	11	85.43±0.55	-9.9%	4.289	0.832	0.075	3/3
τ=0.85	11	83.01±1.84	-12.4%	4.275	0.814	0.084	3/3
τ=0.8	11	79.50±1.32	-16.1%	4.263	0.799	0.096	3/3
RT_9	14	84.11±2.26	-11.3%	4.327	0.855	0.058	3/3
RT_7	16	77.76±1.72	-18.0%	4.335	0.837	0.070	3/3
RT_6	17	80.80±0.39	-14.8%	4.340	0.838	0.073	3/3
RT_5	18	80.22±3.28	-15.4%	4.335	0.833	0.076	3/3
LM1B medium							
released (τ=1)	11	87.47±2.16	–	4.339	0.895	0.043	–
τ=0.9	11	75.46±1.56	-13.7%	4.309	0.870	0.057	3/3
τ=0.85	11	72.40±2.55	-17.2%	4.298	0.859	0.065	3/3
τ=0.8	11	70.22±1.40	-19.7%	4.285	0.848	0.071	3/3
RT_7	16	70.30±1.25	-19.6%	4.341	0.880	0.049	3/3
RT_6	17	69.43±0.78	-20.6%	4.342	0.873	0.056	3/3
RT_6×2	23	62.30±0.25	-28.8%	4.342	0.860	0.062	3/3
LM1B large							
released (τ=1)	11	80.11±0.34	–	4.339	0.895	0.043	–
τ=0.9	11	71.24±0.71	-11.1%	4.313	0.871	0.057	3/3
τ=0.85	11	67.91±1.76	-15.2%	4.298	0.861	0.064	3/3
τ=0.8	11	67.29±0.94	-16.0%	4.286	0.855	0.068	3/3
RT_9	14	75.16±1.68	-6.2%	4.346	0.892	0.045	3/3
RT_7	16	67.10±1.15	-16.2%	4.342	0.885	0.047	3/3
RT_6	17	62.46±1.63	-22.0%	4.338	0.878	0.052	3/3
RT_5	18	63.45±1.15	-20.8%	4.343	0.869	0.060	3/3
RT_6×2	23	56.63±0.88	-29.3%	4.346	0.869	0.059	3/3
RT_5×2	25	59.00±0.27	-26.3%	4.350	0.855	0.068	3/3
RT_4+RT_8	23	61.51±1.26	-23.2%	4.353	0.859	0.067	3/3
RT_6, $\tau_{\geq 6}$=0.9	17	58.97±0.48	-26.4%	4.325	0.868	0.058	3/3

Table 2: LM1B, released small and large checkpoints, authors' FLM protocol (GPT-2 Large), 3 seeds × 256 samples. Diversity measured on BERT word-piece ids. Refinement improves Gen. PPL at equal or higher entropy; temperature cannot.

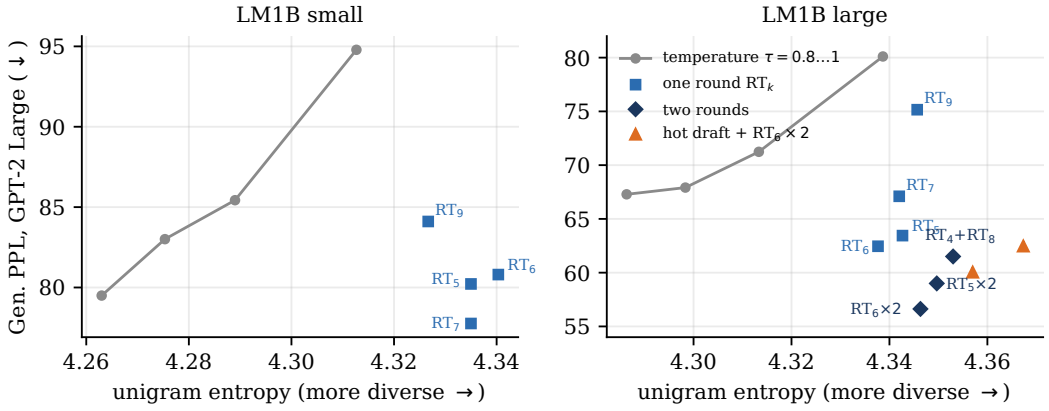


Figure 4: LM1B: Gen. PPL against unigram entropy. Temperature (grey) trades entropy for perplexity; refinement moves straight down, and on the large model two rounds reach −29% at slightly *higher* entropy.

4.339 to 4.286, i.e. by narrowing the vocabulary the samples draw from, whereas RT_6 reaches −22% at entropy 4.338. Two rounds from level 6 reach 56.63 on the large model (−29.3%) at entropy 4.346, higher than the released sampler's. The released medium model behaves the same way (RT_6 −20.6%, RT_6×2 −28.8% at entropy 4.342 vs. 4.339). Across the three sizes the gain of RT_6 grows rather than shrinks, −14.8% (small), −20.6% (medium), −22.0% (large); if the round trip were only compensating for a weak generator, a stronger generator should have less to gain, so this ordering suggests the mismatch it repairs is a property of the training recipe, not of model

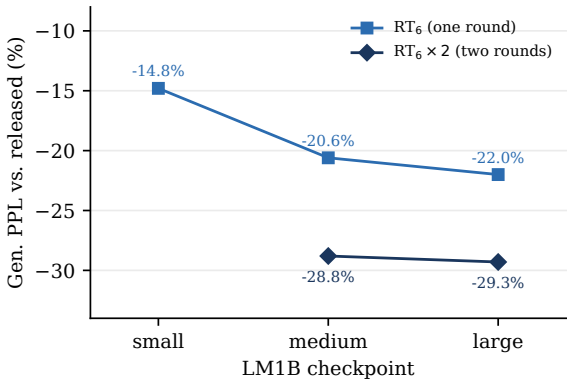


Figure 5: Gain of round-trip refinement against model size on LM1B (same seeds and protocol as Table 2). The gain grows from the small to the large checkpoint; a method that only compensated for a weak generator would show the opposite trend.

capacity. Entropy is the least sensitive diversity axis on LM1B, so we also check the stricter one: measured on distinct-3 of the raw ids, RT_6 is 13–17% and RT_6×2 17–21% below the temperature curve on every seed (1–6), and the hot-draft sampler of Section 5.5 reaches −21% at the released sampler's own distinct-3 (0.894 vs. 0.896).

5.3 Conditional generation

Task	Sampler	NFE	R-1	R-2	R-L	Gen. PPL
XSum	released (τ=1)	8	34.63	12.84	27.77	82.4
XSum	RT_7	10	34.78	12.90	27.84	75.7
XSum	RT_6	11	34.79	12.87	27.89	71.4
XSum	RT_5	12	34.84	12.89	27.90	69.9
XSum	RT_4	13	34.85	12.98	27.93	69.1
XSum	RT_3	14	34.88	13.09	28.10	69.5
XSum	RT_5×2	16	34.57	12.69	27.59	82.5
WMT14	released (τ=1)	10	BLEU 20.51, chrF 48.04			len. 0.90
WMT14	RT_8	13	BLEU 19.05, chrF 47.09			len. 0.98
WMT14	RT_7	14	BLEU 19.01, chrF 47.35			len. 0.94
WMT14	RT_6	15	BLEU 16.38, chrF 46.39			len. 1.07
WMT14	RT_5	16	BLEU 17.21, chrF 46.80			len. 1.01
WMT14	RT_4	17	BLEU 20.45, chrF 47.57			len. 0.95
WMT14	RT_6×2	20	BLEU 17.44, chrF 46.52			len. 1.05

Table 3: Conditional tasks, released checkpoints, greedy decoding as configured, 1,000 test documents. XSum: refinement makes summaries more fluent with a small ROUGE gain; two rounds hurt. WMT14 de–en: refinement lowers BLEU, because the tokenizer round trip itself changes the output.

Summarisation gains, translation does not. The two conditional tasks behave differently, and the difference is informative (Table 3). On XSum, ROUGE measures overlap with the reference summary and Gen. PPL measures how fluent the summary itself reads. One round from level 3 raises ROUGE-1/2/L by +0.25/+0.25/+0.33, a small but consistent gain in content overlap, and lowers the Gen. PPL of the summaries from 82.4 to 69.5 (−16%): the summaries say roughly the same things, but read noticeably better. Two rounds overshoot and lose both (ROUGE-L 27.59, Gen. PPL back at 82.5), so for a conditional task one round is the useful setting. On WMT14 de–en, by contrast, refinement lowers BLEU at every restart level, least for the coarsest restart (RT_4: 20.45 vs. 20.51, a wash) and by over 4 points for RT_6. The sanity check of Section 3 explains why: re-encoding a WMT14 output and decoding it again, with no regeneration at all, already changes 17 of 32 outputs, so on this checkpoint the tokenizer cannot reproduce its own text. The round trip then injects wording changes before the generator even runs, and in translation exact wording is what BLEU rewards. Round-trip refinement therefore needs a tokenizer whose round trip is (near-)lossless on generated text; on TinyStories it is exact, on XSum nearly so (27/32), and on WMT14 it is not.

5.4 Why the round trip helps

Sampled stacks are non-canonical. Figure 8 shows, for 768 TinyStories drafts, the share of codes per level that change when the draft's stack is replaced by the tokenizer's stack for its decoded text. The disagreement is largest where it matters most: 96% of the single level-0 codes change, i.e. the tokenizer almost never picks the "whole-story" code the generator started from; 61–84% change at levels 1–5, the levels that fix the content; and 20–72% at the finer levels, which spell out the words. Yet decoding the re-encoded stack reproduces 751 of the 768 drafts exactly (97.8% of drafts, 99.98% of tokens). So the generator's stack and the tokenizer's stack are two different descriptions of the same text, and the difference is invisible in the output. It is not invisible to the generator: in training, its inputs were always tokenizer stacks (with uniform noise), so at sampling time its fine levels are conditioned on coarse codes of a kind it has never seen. This is the train–test mismatch the round trip removes.

Non-canonicality predicts quality. If the mismatch is what hurts, drafts whose coarse codes the tokenizer rewrites most should be the worst drafts, and the ones the round trip helps most. Table 4 tests this per sample: for each draft we measure its *non-canonicality*, the share of its level 0–5 codes that re-encoding changes, and its own GPT-2 Large perplexity. For the released sampler the two are rank-correlated at ρ=0.37 ($\pm$0.12 across seeds; $p < 10^{-4}$ on every seed), a moderate correlation for a per-sample signal, and it is large enough to matter in practice: the fifth of drafts rewritten least has Gen. PPL 13.6, the fifth rewritten most 20.1, a 48% spread in quality predicted by the tokenizer alone. RT_6 lowers the perplexity of 92% of individual stories, so the average gains above are not carried by a few samples, and its per-story gain is itself correlated with non-canonicality (ρ=0.28): the least-rewritten fifth moves from 13.6 to 11.9, the most-rewritten from 20.1 to 15.1, i.e. the worst drafts gain almost three times as much as the best. On TinyStories, non-canonicality is thus a free, tokenizer-side quality signal for a draft, available before any reference model is consulted, and the drafts it flags are the ones the round trip repairs most. On LM1B large the signal is weak, and the reason is visible in the numbers: nearly every draft is rewritten 85–97% (mean 0.86 against 0.76 on TinyStories, where drafts range from 0.46 to 0.90), so there is little variation between drafts for a correlation to pick up. The rank correlation is only 0.10 $\pm$ 0.09 and the fifths' draft perplexities rise only from 76.4 to 84.7; RT_6 still improves 88% of sequences, by amounts that barely depend on the score (ρ=0.09). The score separates drafts where non-canonicality varies between them, not where it is uniformly high; the repair works in both cases.

Sampler	non-canon.	ρ(non-canon., log PPL)	ρ(non-canon., gain)	improved
TinyStories (GPT-2 Large, per story)				
released (τ=1)	0.755	+0.371$\pm$0.122	–	–
RT_6	0.758	+0.379$\pm$0.052	+0.282$\pm$0.004	92%
best-of-4+RT_6	0.692	+0.203$\pm$0.044	+0.060$\pm$0.046	90%
best-of-8+RT_6	0.650	+0.046$\pm$0.065	+0.041$\pm$0.039	89%
RT_6 on drafts with non-canonicality 0.47–0.72		draft 13.62 → 11.87		87%
RT_6 on drafts with non-canonicality 0.72–0.75		draft 15.05 → 12.61		91%
RT_6 on drafts with non-canonicality 0.75–0.78		draft 15.95 → 13.33		92%
RT_6 on drafts with non-canonicality 0.78–0.81		draft 17.79 → 14.09		95%
RT_6 on drafts with non-canonicality 0.81–0.90		draft 20.14 → 15.09		96%
LM1B large (GPT-2 Large, per sequence)				
released (τ=1)	0.859	+0.098$\pm$0.087	–	–
RT_6	0.865	+0.108$\pm$0.032	+0.092$\pm$0.065	88%
best-of-4+RT_6	0.774	+0.033$\pm$0.139	+0.079$\pm$0.038	88%
RT_6 on drafts with non-canonicality 0.66–0.77		draft 76.40 → 61.99		83%
RT_6 on drafts with non-canonicality 0.77–0.88		draft 78.12 → 61.11		90%
RT_6 on drafts with non-canonicality 0.88–0.91		draft 79.79 → 62.34		85%
RT_6 on drafts with non-canonicality 0.91–0.93		draft 81.52 → 61.71		90%
RT_6 on drafts with non-canonicality 0.93–0.97		draft 84.67 → 65.34		91%

Table 4: Per-sample non-canonicality (share of level 0–5 codes changed by re-encoding the draft) against per-sample Gen. PPL, seeds 1–3 (3×256; Spearman, mean $\pm$ sd over seeds). "gain" is the drop in log PPL from the draft to the refined text, "improved" the share of samples whose perplexity the round trip lowers. The indented rows bin the RT_6 drafts into fifths by non-canonicality and give draft → refined Gen. PPL per fifth (geometric means over samples, averaged over seeds). Best-of-N rows score the selected drafts.

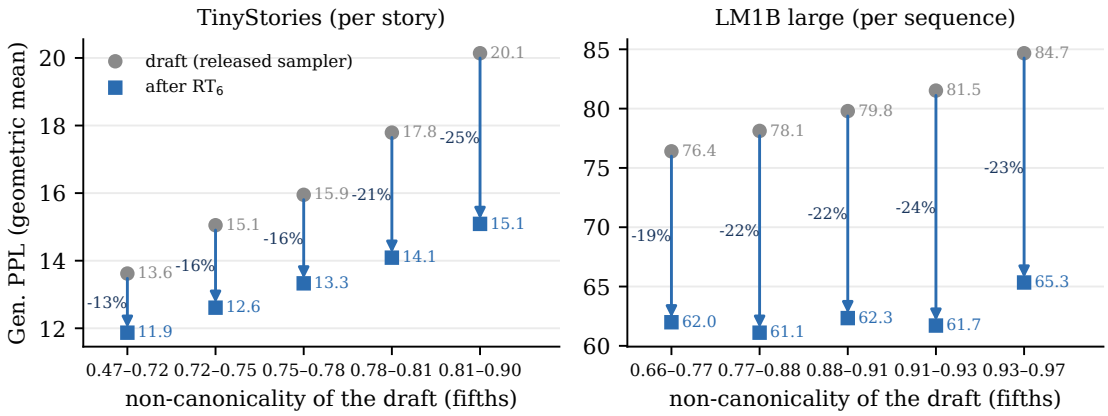


Figure 6: Per-fifth view of Table 4: drafts binned by how much of their coarse stack the tokenizer rewrites, with their Gen. PPL before and after one round from level 6. On TinyStories the most-rewritten fifth starts 48% worse than the least-rewritten and gains almost twice as much; on LM1B large every fifth is rewritten heavily and gains about the same.

Selecting drafts by canonicality is a diversity trade. The signal suggests a selector: draw N drafts and keep, per sample, the one re-encoding changes least (Table 5). At first sight it works: best-of-8 lowers Gen. PPL by 12.9%, almost as much as RT_6 (−17.3%), although for 8× the passes. The diversity columns show what it actually does. By picking the drafts the tokenizer finds least surprising, it picks the most generic ones: distinct-3 falls from 0.749 to 0.699 and cross-sample overlap rises from 0.162 to 0.216, a larger loss of diversity than τ=0.8 incurs, and at that diversity the temperature curve is 11.7% *better* than the selector. In other words, best-of-N by canonicality is a worse version of lowering the temperature. Refining the selected draft recovers part of the loss (best-of-4 + RT_6: −25.3% raw, but only −4.5% against the frontier, versus −12.4% for RT_6 alone), and the larger selectors fall below the diversity range of the curve altogether. On LM1B large, where non-canonicality barely varies between drafts, the selector does nothing at all: best-of-4 scores 79.7 against 80.1 for the released sampler and adds 0.8% to RT_6 (62.0 vs. 62.5). Non-canonicality therefore ranks drafts by quality but is not a free lunch as a filter; regenerating the fine levels on canonical coarse codes is what improves a draft without trading its diversity away.

Sampler	NFE	Gen. PPL ↓	Δ	non-canon.	dist-3 ↑	xs-4 ↓	vs. τ frontier
released (τ=1)	11	16.15±0.17	–	0.755	0.749	0.162	–
τ=0.8	11	13.34±0.25	-17.4%	–	0.719	0.191	–
RT_6	17	13.36±0.12	-17.3%	0.758	0.740	0.170	-12.4%
RT_5×2	25	11.70±0.31	-27.6%	0.754	0.718	0.191	-12.9%
RT_6, draft τ=1.1	17	14.15±0.21	-12.4%	0.793	0.753	0.156	-14.7%
best-of-4	44	14.52±0.52	-10.1%	0.687	0.711	0.203	+11.7%
best-of-8	88	14.08±0.14	-12.9%	0.652	0.699	0.216	+11.7%
best-of-4+RT_6	50	12.07±0.31	-25.3%	0.692	0.700	0.212	-4.5%
best-of-8+RT_6	94	11.90±0.07	-26.3%	0.650	0.692	0.219	–
best-of-8+RT_5×2	102	10.98±0.12	-32.0%	0.651	0.681	0.230	–
LM1B large (FLM protocol; last column against the temperature curve at equal entropy)							
released (τ=1)	11	80.11±0.34	–	0.859	0.895	0.043	–
RT_6	17	62.46±1.63	-22.0%	0.865	0.878	0.052	-20.9%
best-of-4	44	79.69±1.84	-0.5%	0.774	0.894	0.044	+0.5%
best-of-4+RT_6	50	61.96±2.14	-22.6%	0.774	0.875	0.053	-25.1%

Table 5: Best-of-N drafts selected by canonicality, TinyStories seeds 1–3 (GPT-2 Large, 3 × 256). "non-canon." is the mean non-canonicality of the (selected) drafts. The frontier column is missing where a sampler is less diverse than any point of the temperature curve ($\tau \geq 0.7$).

The re-encoding is what helps. Table 11 separates the ingredients. The first control asks whether the extra passes alone help. Regenerating levels 6–11 conditioned on the draft's own coarse codes ($Resample_6$) costs the same six passes as RT_6 but gives no gain at all: 16.43 against 16.15 on TinyStories and +0.6% on LM1B, both

within seed noise. This is expected, since it only draws the fine levels again from the conditional distribution that produced them, and it rules out the simplest explanation, that any second attempt at the fine levels is better than the first. The second control asks whether the text itself matters, or only re-quantization. Re-quantizing the draft's latent with Eq. (1) instead of re-encoding its text (LatentRQ$_6$, a final-latent analogue of image scale-travel, which re-quantizes a partial map at an intermediate scale) changes the coarse codes almost as much, except level 0, which it keeps 72% of the time, and recovers part of the gain: −11.5% on TinyStories and −17.9% on LM1B. But it also lowers diversity, so at matched diversity it is worth −6.8% and −7.7%, against −12.4% and −20.9% for RT$_6$. Going through the decoded text, which discards whatever in the latent does not affect the output and lets the encoder describe the text afresh, roughly doubles the matched-diversity gain; the discrete text is a better anchor than the continuous latent. The same ordering holds with two rounds (Resample$_5$×2 −0.2%, LatentRQ$_5$×2 −19.9%, RT$_5$×2 −27.6%), so it is not a one-round accident.

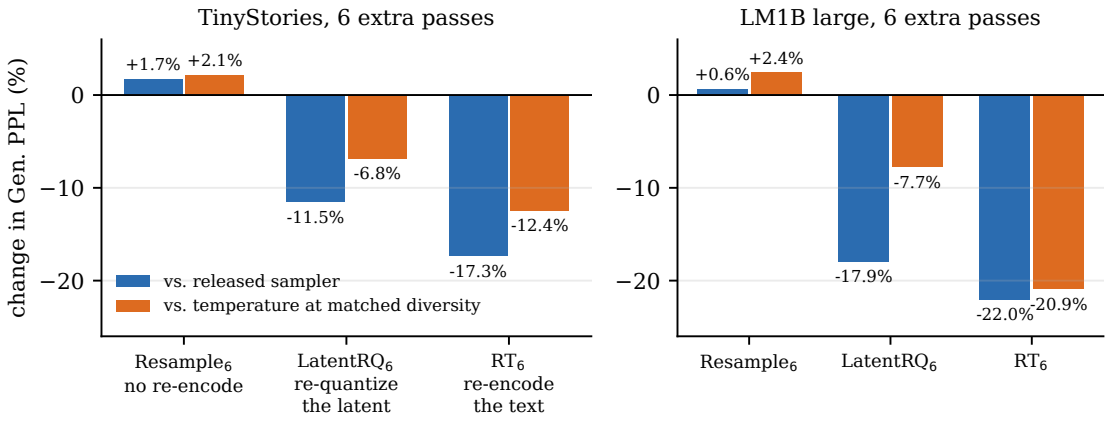


Figure 7: Three ways to spend the same six extra passes (Table 11). Resampling the fine levels from the draft's own coarse codes gives nothing; re-quantizing the latent recovers part of the gain but loses diversity; re-encoding the decoded text gives the full gain at almost unchanged diversity. Orange bars compare each sampler with the released sampler's temperature curve at the same diversity.

5.5 Spending the gain on diversity

Hot drafts. Because refinement lowers Gen. PPL at almost constant diversity, the gain can be spent differently: start from a *hotter* draft, which is more diverse but worse, and let the round trip repair the damage (Table 11, Figure 2). A draft at τ=1.1 on its own is 16% worse than the released sampler (18.76), and at τ=1.2 49% worse (24.01). With the draft at τ=1.1 and one round from level 6 at τ=1, Gen. PPL is 14.15 (−12.4% against the released sampler, on all seeds) while distinct-3 rises from 0.749 to 0.753, cross-sample overlap falls from 0.162 to 0.156, unigram entropy is unchanged (3.981) and doubled words fall from 0.39% to 0.26%. This sampler is better than the released one on every axis we measure, for six extra passes, which no temperature setting can achieve since temperature always trades one axis for the other. With the draft at τ=1.2 and the regenerated levels at τ=0.8, samples are clearly more diverse than the released sampler's (distinct-3 0.766, overlap 0.143) and still 7.3% more fluent; the temperature curve at that diversity is at 19.2, so the frontier gain is 22%, the largest in the paper. Hot drafts with two or three rounds reach −22% to −27% Gen. PPL at 17–18% below the frontier. On LM1B large the same recipe works: a τ=1.2 draft with RT$_6$×2 gives 62.47 (−22.0%) at higher entropy (4.367 vs. 4.339), the same distinct-3 (0.894 vs. 0.895) and lower cross-sample overlap (0.040 vs. 0.043), again better on every axis. The hot-draft settings come from a small grid (draft $\tau \in \{1.1, 1.2\}$, three round schedules) chosen on seeds 1–3 and confirmed on held-out seeds below. Part of these gains comes from the per-level temperature schedule itself rather than from the round trip; Section 5.9 separates the two with exact 11-pass twins.

5.6 Held-out seeds

Replication with nothing re-tuned. Every TinyStories setting above (restart level, number of rounds, temperatures) was chosen while looking at seeds 1–3, so the gains reported there could in part be selection effects. Table 6 therefore re-runs the main samplers, the ablations and the full temperature curve on three fresh seeds (4–6, 3×256 samples) with nothing re-tuned. Three things should hold if the effect is real, and all three do. First, the raw gains are unchanged: RT$_6$ lowers Gen. PPL from 16.32 to 13.23 (−18.9%, against −17.3% on seeds 1–3), and three

rounds reach 11.50 (−29.5%). Second, the gain survives the diversity control. Lowering the temperature to τ=0.8 also reaches −17.6%, but only by making the samples more alike (distinct-3 falls from 0.748 to 0.717 and cross-sample overlap rises from 0.165 to 0.195); RT_6 keeps distinct-3 at 0.737 and overlap at 0.173, so at its diversity the temperature curve would give a perplexity 14.5% higher than RT_6 achieves. This matched-diversity margin is the quantity that cannot be obtained by turning the temperature knob, and it is the same size as on seeds 1–3 (10–14%) or slightly larger: 15.3% for RT_5×2. The best sampler, a hot τ=1.2 draft refined twice with the regenerated levels at τ=0.8, is 23.6% below the released sampler at *the same* diversity (distinct-3 0.742 vs. 0.748, overlap 0.165 vs. 0.165), which puts it 21.4% beyond the frontier. Third, the ablation still separates the mechanism from the extra compute: resampling the fine levels without re-encoding changes nothing (−1.2%, within seed noise), so the six additional passes are worthless unless they start from the re-encoded coarse levels. LM1B large behaves the same way: held-out seeds give −20.2% for RT_6 and −27.1% for RT_6×2, against −22.0% and −29.3% on seeds 1–3, a difference of two points that is the expected seed-to-seed variation.

Sampler	NFE	Gen. PPL ↓	Δ	dist-3 ↑	xs-4 ↓	vs. τ frontier
released (τ=1)	11	16.32±0.19	–	0.748	0.165	–
τ=0.8	11	13.44±0.22	-17.6%	0.717	0.195	–
RT_6	17	13.23±0.08	-18.9%	0.737	0.173	-14.5%
RT_5×2	25	11.94±0.11	-26.8%	0.725	0.185	-15.3%
RT_3+RT_6+RT_9	29	11.50±0.10	-29.5%	0.715	0.199	-14.3%
RT_6, $\tau_{\geq 6}$=0.8	17	12.98±0.10	-20.5%	0.735	0.174	-14.8%
RT_6, draft τ=1.1	17	13.96±0.04	-14.5%	0.746	0.166	-13.5%
RT_6, draft τ=1.2, $\tau_{\geq 6}$=0.8	17	14.46±0.08	-11.4%	0.755	0.153	-17.7%
RT_5×2, draft τ=1.2, $\tau_{\geq 6}$=0.8	25	12.47±0.57	-23.6%	0.742	0.165	-21.4%
$Resample_6$	17	16.12±0.35	-1.2%	0.747	0.164	-0.7%
$LatentRQ_6$	17	14.27±0.08	-12.5%	0.739	0.172	-8.5%

Table 6: TinyStories on held-out seeds 4–6 (GPT-2 Large, 3×256). The frontier is the temperature curve (τ=0.7..1.2) measured on the same seeds.

5.7 Beyond generative perplexity: an LLM judge and MAUVE

Two further measures. Gen. PPL rewards text that GPT-2 finds predictable. Two further measures check that refined stories are actually better and closer to human writing. First, an open-weights LLM judge (Qwen2.5-7B-Instruct; 26) compares paired stories (sample i of two samplers starts from the same level-0 draw) in both orders and gives the probability that each is more fluent, grammatical and coherent; averaging the two orders removes its strong preference for the story shown first (0.78–0.91). Second, MAUVE [22] measures the gap between each sampler's 768 stories and 768 human-written validation openings in GPT-2 Large feature space.

X	Y	P(Y preferred)	95% CI	both orders Y / X (%)	note
released	RT_6	0.634	[0.617, 0.651]	30.6 / 3.3	main comparison
released	RT_3+RT_6+RT_9	0.587	[0.573, 0.601]	19.4 / 2.1	
released	RT_6, draft τ=1.1	0.531	[0.518, 0.544]	11.1 / 5.2	more diverse than released
τ=0.8	$RT_5 \times 2$	0.529	[0.515, 0.542]	11.8 / 6.0	equal distinct-3
τ=0.8	RT_6	0.506	[0.493, 0.519]	8.7 / 7.8	equal Gen. PPL
$LatentRQ_6$	RT_6	0.557	[0.539, 0.574]	20.7 / 9.8	text vs. latent round trip
released	τ=0.8	0.537	[0.523, 0.551]	12.6 / 6.0	temperature alone

Table 7: Blind pairwise preferences of Qwen2.5-7B-Instruct over 768 paired TinyStories samples (seeds 1–3), averaged over both presentation orders. "Both orders": share of pairs where the judge prefers the same story in both orders.

What the judge and MAUVE say. A preference of 0.5 would mean the judge cannot tell two samplers apart. It prefers RT_6 over the released sampler at 0.634, and the stricter count tells the same story: it picks the RT_6 story in *both* presentation orders for 30.6% of pairs and the released story in both orders for only 3.3%, a nine-to-one margin among the pairs where it is sure. Three rounds are likewise preferred (0.587), and the hot-draft sampler, which is more diverse than the released one, still wins (0.531, interval above 0.5), so the perplexity gains correspond to

stories a reader would rank higher, not only to text GPT-2 finds predictable. The judge also reproduces the two controls. At equal distinct-3, two rounds beat the temperature-lowered sampler (0.529, interval above 0.5), and the text round trip beats latent re-quantization (0.557). At equal Gen. PPL, however, RT_6 and τ=0.8 tie (0.506): once two samplers reach the same perplexity, the judge sees individual stories of the same quality, and the advantage of refinement over temperature at that point is the diversity it keeps across stories, which a pairwise judge cannot see. MAUVE (Table 8), which compares the distribution of 768 stories with that of 768 human openings, rises from 0.882 for the released sampler to 0.94–0.955 for refinement, best for $RT_5\times2$; the refined samples are therefore closer to human text as a population, not only sentence by sentence. Lowering the temperature also raises MAUVE (0.943–0.946), so MAUVE confirms the direction of the gain but does not by itself separate refinement from temperature; the frontier and the twins do.

Sampler	MAUVE ↑	Gen. PPL	dist-3
released (τ=1)	0.882±0.012	16.15	0.749
τ=0.9	0.946±0.002	14.64	0.735
τ=0.8	0.943±0.011	13.34	0.719
Resample_6	0.913±0.004	16.43	0.750
LatentRQ_6	0.925±0.005	14.29	0.741
RT_6	0.941±0.004	13.36	0.740
RT_6, $\tau_{\geq6}$=0.8	0.946±0.013	12.96	0.739
$RT_5\times2$	0.955±0.005	11.70	0.718
RT_3+RT_6+RT_9	0.945±0.006	11.17	0.708
RT_6, draft τ=1.1	0.929±0.005	14.15	0.753
RT_6, draft τ=1.2, $\tau_{\geq6}$=0.8	0.952±0.001	14.98	0.766

Table 8: MAUVE against 768 human TinyStories openings (GPT-2 Large features, mean ± s.d. over three k-means seeds; differences between sample sets of about 0.03 are within sampling noise).

A second scorer. Gen. PPL depends on the reference model. Table 9 re-scores the main samplers with a second, larger and unrelated reference model, Qwen2.5-7B (base weights, bf16; 26), under the same two protocols. If the gains were an artefact of what GPT-2 Large happens to find predictable, a different scorer would disagree. It does not: the relative gains agree to within a few points on every set, RT_6 −17.8% under Qwen against −17.3% under GPT-2 on TinyStories, three rounds −31.8% against −29.5% on held-out seeds, RT_6 −22.9% against −22.0% on LM1B large and $RT_6\times2$ −31.9% against −28.8% on LM1B medium. Where the two differ, Qwen rates refinement slightly *more* favourably. The twin without the round trip is worse under both scorers (+12.0% and +10.2%), and human text stays far below every sampler (4.70 against 10.05 for RT_6 under Qwen), so neither scorer is saturated. The one disagreement is LM1B small, where GPT-2 Large rates τ=0.8 slightly above RT_6 (−16.1% vs. −14.8%) and Qwen rates RT_6 clearly above it (−18.4% vs. −11.3%); this is the comparison that ignores diversity, and the diversity-matched comparisons of Table 2, which favour refinement, do not depend on the scorer.

5.8 Reasoning: Sudoku

Greedy refinement. NCM's Sudoku results [13] make it a test of whether refinement only polishes surface text or also helps a task with a single correct answer. On the released Sudoku-hard model (1,000 test puzzles, greedy decoding as configured), every restart level solves more puzzles than the released sampler (Table 10): from 84.1% to 85.5% with RT_6 and 85.9% with three rounds. These are small differences, so the per-puzzle counts matter more than the percentages: RT_6 newly solves 14 puzzles and breaks none, three rounds solve 19 and break 1. Because the same 1,000 puzzles are scored under both samplers, an exact McNemar test on these counts gives $p=1.2\times10^{-4}$ for RT_6, so a 14-to-0 split is very unlikely to be noise even though 1.4 points is. The gain is small for a reason visible in the codes: here the round trip changes only 2–20% of the codes per level, far less than the 61–96% on stories, because a Sudoku board leaves the tokenizer little freedom, and greedy rounds converge quickly ($RT_6\times2$ and $RT_6\times4$ are identical). Refinement on this difficulty is therefore a small, essentially regression-free gain, not a large one. It does not generalise across difficulties: on Sudoku-easy there is nothing left to fix (99.6% to 99.7%), and on Sudoku-medium it lowers the solve rate from 96.7% to 95.9% (RT_6) and 96.0% (three rounds), so on that checkpoint the round trip breaks slightly more boards than it repairs.

Verifier-guided search. Sudoku also has an exact, free verifier (rows, columns, boxes and the given clues), so extra passes can be spent until a board verifies. We compared, at equal mean generator passes per puzzle, two ways

Sampler	NFE	GPT-2 Large	Δ	Qwen2.5-7B	Δ
TinyStories, seeds 1–3					
released (τ=1)	11	16.15	–	12.28	–
RT_6	17	13.36	-17.3%	10.09	-17.8%
RT_5×2	25	11.70	-27.6%	8.67	-29.4%
RT_6, draft τ=1.1	17	14.15	-12.4%	10.84	-11.7%
TinyStories, held-out seeds 4–6					
released (τ=1)	11	16.32	–	12.43	–
τ=0.8	11	13.44	-17.6%	10.09	-18.8%
RT_6	17	13.23	-18.9%	10.05	-19.1%
RT_5×2	25	11.94	-26.8%	8.92	-28.3%
RT_3+RT_6+RT_9	29	11.50	-29.5%	8.48	-31.8%
RT_5×2, draft τ=1.2, $\tau_{\geq 6}$=0.8	25	12.47	-23.6%	9.43	-24.1%
TinyStories twins and human text, seeds 4–6					
RT_5×2, draft τ=1.3, $\tau_{\geq 6}$=0.8	25	12.96	-20.6%	9.78	-21.3%
$\tau_{\geq 1}$=1.3, $\tau_{\geq 5}$=1, $\tau_{\geq 6}$=0.8	11	17.99	+10.2%	13.91	+12.0%
human text	0	8.51	-47.9%	4.70	-62.2%
human + RT_6	6	9.39	-42.4%	5.54	-55.4%
LM1B small					
released (τ=1)	11	94.79	–	78.82	–
τ=0.8	11	79.50	-16.1%	69.91	-11.3%
RT_6	17	80.80	-14.8%	64.34	-18.4%
LM1B medium					
released (τ=1)	11	87.47	–	67.17	–
RT_6	17	69.43	-20.6%	52.03	-22.5%
RT_6×2	23	62.30	-28.8%	45.75	-31.9%
LM1B large					
released (τ=1)	11	80.11	–	59.62	–
τ=0.8	11	67.29	-16.0%	50.86	-14.7%
RT_6	17	62.46	-22.0%	45.94	-22.9%
LM1B large, seeds 4–6					
released (τ=1)	11	79.87	–	61.14	–
RT_6×2	23	58.22	-27.1%	42.56	-30.4%
RT_6×2, draft τ=1.2	23	63.37	-20.7%	46.32	-24.3%
$\tau_{\geq 1}$=1.2, $\tau_{\geq 6}$=1	11	90.27	+13.0%	67.74	+10.8%

Table 9: Gen. PPL under two reference models (means over three seeds; TinyStories protocol or, for LM1B, the FLM protocol with the first 128 tokens of each scorer's tokenizer). Δ is relative to the released sampler of the same block.

of spending that budget: retrying from the current board with a round trip at temperature T, or discarding it and drawing a fresh sample. Fresh sampling at τ=1 is clearly better at every budget: 89.9% solved at 16.8 passes per puzzle, 91.0% at 19.2 and 93.4% at 23.9, against at most 89.5–90.7% for round-trip retries at 19–22 passes. The reason is that a round-trip retry keeps most of the wrong board (the coarse plan is re-encoded from it), so repeated retries stay correlated with the same mistake, whereas a fresh sample makes an independent attempt. Resampling only the coarse plan with greedy fine levels does not beat fresh sampling either (best 92.3–93.3% at about 24 passes over seeds 1, 4, 5 and 6). The practical conclusion: NCM's Sudoku accuracy rises from 84% to 93% with verifier-guided test-time compute, but plain resampling, not round-trip refinement, is the way to spend it when an exact verifier exists.

5.9 Is it a temperature schedule? Exact twins and human text

Exact twins. Because attention is block-diagonal, level j sees only $\hat{z}_{j-1}$, so resampling levels $\geq k$ from the draft's own coarse codes is distributed exactly like the draft. Every refinement recipe therefore has an exact 11-pass twin: the same per-level temperatures, no round trip. Table 12 compares recipes with their twins on held-out seeds 4–6, together with a temperature curve from τ=0.6 to 1.3 measured on the same seeds.

Sampler	NFE	solved %	cell %	new / broken	McNemar p
released (greedy)	11	84.1	98.13	–	–
RT_9	14	84.7	98.14	6 / 0	3.1e-02
RT_8	15	84.9	98.15	8 / 0	7.8e-03
RT_6	17	85.5	98.19	14 / 0	1.2e-04
RT_4	19	84.9	98.15	10 / 2	3.9e-02
RT_6×2	23	85.2	98.20	11 / 0	9.8e-04
RT_6×4	35	85.2	98.21	11 / 0	9.8e-04
RT_4×3	35	85.0	98.15	10 / 1	1.2e-02
RT_3+RT_6+RT_9	29	85.9	98.21	19 / 1	4.0e-05

Table 10: Sudoku-hard, released checkpoint, greedy decoding, 1,000 test puzzles. "New / broken": puzzles solved only by the refined sampler / only by the released one.

Sampler	NFE	Gen. PPL ↓	Δ	H	dist-3 ↑	xs-4 ↓	vs. τ frontier
released (τ=1)	11	16.15±0.17	–	3.980	0.749	0.162	–
$Resample_6$	17	16.43±0.33	+1.7%	3.980	0.750	0.164	+2.1%
$LatentRQ_6$	17	14.29±0.25	-11.5%	3.966	0.741	0.170	-6.8%
RT_6	17	13.36±0.12	-17.3%	3.974	0.740	0.170	-12.4%
$Resample_5$×2	25	16.13±0.27	-0.2%	3.982	0.745	0.171	+3.7%
$LatentRQ_5$×2	25	12.94±0.25	-19.9%	3.963	0.725	0.188	-6.7%
RT_5×2	25	11.70±0.31	-27.6%	3.970	0.718	0.191	-12.9%
τ=1.1	11	18.76±0.32	+16.1%	3.993	0.764	0.151	–
τ=1.2	11	24.01±0.95	+48.7%	3.998	0.787	0.132	–
RT_6, draft τ=1.1	17	14.15±0.21	-12.4%	3.981	0.753	0.156	-14.7%
RT_6, draft τ=1.2	17	15.28±0.30	-5.4%	3.979	0.767	0.143	-20.9%
RT_6, draft τ=1.2, $\tau_{\geq 6}$=0.8	17	14.98±0.24	-7.3%	3.980	0.766	0.143	-22.0%
RT_5×2, draft τ=1.1	25	12.10±0.36	-25.1%	3.974	0.733	0.177	-16.7%
RT_5×2, draft τ=1.2	25	12.63±0.19	-21.8%	3.973	0.740	0.167	-17.1%
RT_5×2, draft τ=1.2, $\tau_{\geq 6}$=0.8	25	12.45±0.11	-22.9%	3.972	0.740	0.167	-18.1%
RT_3+RT_6+RT_9, draft τ=1.2	29	11.80±0.33	-27.0%	3.964	0.727	0.184	-17.3%
LM1B large (FLM protocol)							
released (τ=1)	11	80.11±0.34	–	4.339	0.895	0.043	–
$Resample_6$	17	80.60±2.00	+0.6%	4.339	0.896	0.043	+2.4%
$LatentRQ_6$	17	65.78±0.54	-17.9%	4.313	0.874	0.055	-7.7%
RT_6	17	62.46±1.63	-22.0%	4.338	0.878	0.052	-20.9%
RT_6×2	23	56.63±0.88	-29.3%	4.346	0.869	0.059	–
RT_6×2, draft τ=1.1	23	60.04±1.24	-25.0%	4.357	0.881	0.049	–
RT_6×2, draft τ=1.2	23	62.47±0.37	-22.0%	4.367	0.894	0.040	–

Table 11: Ablations and hot drafts (TinyStories, GPT-2 Large, 3 seeds × 256; LM1B large, FLM protocol, same seeds). Resample: regenerate without re-encoding. LatentRQ: re-quantize the latent instead of re-encoding the text. Draft τ: temperature of the whole draft; $\tau_{\geq 6}$: temperature of levels 6–11 (of the rounds only when a draft temperature is given, otherwise of the draft and the rounds). Last column: Gen. PPL relative to the temperature curve (τ=0.7..1.2 on TinyStories, 0.8..1 on LM1B) at the same distinct-3 (TinyStories) or entropy (LM1B); "–" where a variant lies outside the curve.

What the twins show. The twin test asks a sharp question: if we keep every temperature of a recipe but skip the round trip, how much of the gain remains? At the released temperature the twin of RT_6 is the released sampler itself, so refinement accounts for the whole −14.5% matched-diversity gain. With hot coarse and cool fine levels the answer changes, and this is the one place where part of our gain is not the round trip: the per-level schedule alone, with no round trip at all, already lies well below the global-temperature curve (−21.0% for the twin of the best recipe, −11.3% for the twin of the τ=1.2 recipe). Heating the coarse levels and cooling the fine ones is itself a better sampler than a single global temperature, a result not reported in the NCM paper. Refinement adds 4–10 points on top (−25.2% and −21.4% against the frontier). We therefore report the hot-draft samplers as "schedule plus round trip" and attribute to refinement only its increment. On LM1B large the schedule alone does not help: the twin of the hot-draft sampler is far worse at a similar entropy (90.27 vs. 63.37, entropy 4.361 vs. 4.368), so there the whole gain is the round trip.

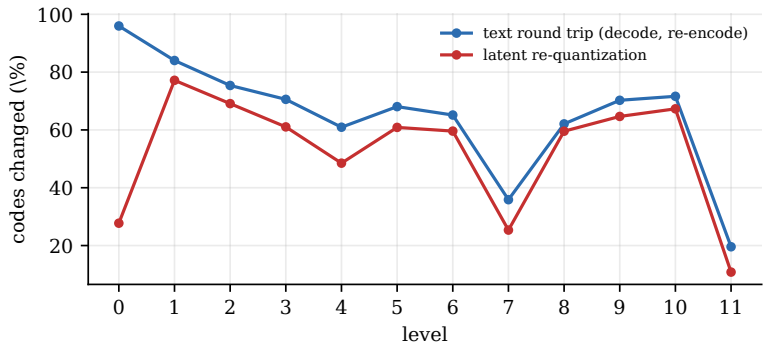


Figure 8: Share of codes changed per level when the draft's stack is replaced by the tokenizer's stack for the same text (blue) or by the re-quantized latent (red), TinyStories, 768 samples.

Sampler	NFE	Gen. PPL ↓	dist-3 ↑	xs-4 ↓	vs. τ frontier
released (τ=1)	11	16.32	0.748	0.165	–
RT_6	17	13.23	0.737	0.173	-14.5%
RT_5×2, draft τ=1.3, $\tau_{\geq 6}$=0.8 (best recipe)	25	12.96	0.754	0.153	-25.2%
$\tau_{\geq 1}$=1.3, $\tau_{\geq 5}$=1, $\tau_{\geq 6}$=0.8 (its twin)	11	17.99	0.778	0.133	-21.0%
RT_5×2, draft τ=1.2, $\tau_{\geq 6}$=0.8 (recipe)	25	12.47	0.742	0.165	-21.4%
$\tau_{\geq 1}$=1.2, $\tau_{\geq 5}$=1, $\tau_{\geq 6}$=0.8 (its twin)	11	17.10	0.764	0.147	-11.3%
RT_6, $\tau_{\geq 1}$=1.2, $\tau_{\geq 6}$=0.8 (recipe)	17	14.04	0.752	0.155	-17.1%
$\tau_{\geq 1}$=1.2, $\tau_{\geq 6}$=0.8 (its twin)	11	17.73	0.766	0.146	-10.2%
τ=0.6	11	12.10	0.681	0.232	–
τ=0.7	11	12.65	0.699	0.214	–
τ=0.8	11	13.44	0.717	0.195	–
human text	0	8.51	0.775	0.139	–
human + RT_6	6	9.39	0.772	0.141	–
human + RT_5×2	14	10.35	0.760	0.151	–
human + RT_3+RT_6+RT_9	18	12.12	0.750	0.163	–

Table 12: Refinement recipes and their exact 11-pass twins (same per-level temperatures, no round trip), TinyStories held-out seeds 4–6. "$\tau_{\geq k}$" sets the temperature of levels $\geq k$; parts apply in order. Human rows apply the rounds to 768 real validation openings instead of a draft.

Human text. The human-text test asks a different question: is the round trip a general "sharpener" that makes any text more predictable, or does it specifically repair the generator's drafts? Applied to 768 real human openings instead of drafts, refinement does not lower Gen. PPL below that of the human text: one round raises it from 8.51 to 9.39, with distinct-3 and cross-sample overlap essentially unchanged (0.772 vs. 0.775, 0.141 vs. 0.139), and further rounds raise it more (10.35, 12.12). A sharpening operator would have pushed human text to lower perplexity and lower diversity. Instead the round trip leaves real text close to where it is, and the small rise is the generator's own realisation errors when it regenerates the fine levels. The gain on drafts is therefore a repair of something specific to the drafts, the non-canonical coarse codes, not a bias of the procedure that any input would show.

5.10 Adaptive rounds: spend passes only where the draft drifts

Two gates. Non-canonicality is measured by the same encoder pass that the round trip needs anyway, so it is free to use as a gate. We test two uses of it (Table 13, Figure 9). *Gated round* ($ART_k(\theta)$): run the round from level k only for the samples whose draft the re-encode rewrites more than a share θ of the level 0–5 codes; the others keep their draft and spend no extra passes. *Early stop* ($ART_k{\times}n(\theta)$): repeat up to n rounds, applying the same test to the current text before each round. The reported cost is the mean number of generator passes per sample. The per-round non-canonicality explains why the two gates need different thresholds: drafts are rewritten at a median of 0.76 (TinyStories) and 0.89 (LM1B), but after one round the kept levels *are* the tokenizer's, so the text that comes out is rewritten far less, at a median of 0.44 after RT_5 and 0.18 after RT_6 on TinyStories (0.53 after RT_6 on LM1B), and less again after a second round (0.28 and 0.14). A threshold tuned for drafts therefore never fires a

second round; the early-stop thresholds below are set in the range where the second and third rounds actually vary.

Sampler	passes	refined	Gen. PPL ↓	Δ	dist-3 ↑	xs-4 ↓	vs. τ frontier
TinyStories: one round, gated by the draft's non-canonicality							
released (τ=1)	11	–	16.15±0.17	–	0.749	0.162	–
RT_6	17	100%	13.36±0.12	-17.3%	0.740	0.170	-12.4%
$ART_6(\theta$=0.70)	16.1	86%	13.61±0.13	-15.7%	0.741	0.169	-11.1%
$ART_6(\theta$=0.75)	14.5	59%	14.19±0.14	-12.2%	0.740	0.171	-7.0%
$ART_6(\theta$=0.80)	12.4	23%	15.14±0.23	-6.3%	0.745	0.167	-3.4%
TinyStories: up to three rounds, each gated by the current text's non-canonicality							
$RT_5\times2$	25	100%	11.70±0.31	-27.6%	0.718	0.191	-12.9%
$RT_5\times3$	32	100%	11.38±0.05	-29.6%	0.721	0.184	-16.8%
$ART_5\times3(\theta$=0.30)	26.0	100%	11.84±0.21	-26.7%	0.721	0.187	-12.7%
$ART_5\times3(\theta$=0.40)	24.0	100%	12.02±0.15	-25.6%	0.726	0.184	-14.2%
$ART_5\times3(\theta$=0.50)	20.8	99%	12.24±0.22	-24.2%	0.728	0.180	-14.0%
$ART_5\times3(\theta$=0.60)	18.1	97%	12.77±0.30	-20.9%	0.730	0.182	-11.6%
$ART_6\times3(\theta$=0.40)	18.8	100%	12.80±0.38	-20.8%	0.734	0.177	-12.5%
$ART_6\times3(\theta$=0.50)	17.2	99%	13.26±0.28	-17.9%	0.741	0.169	-13.2%
LM1B large (FLM protocol; last column at equal entropy)							
released (τ=1)	11	–	80.11±0.34	–	0.895	0.043	–
RT_6	17	100%	62.46±1.63	-22.0%	0.878	0.052	-20.9%
$ART_6(\theta$=0.80)	15.3	71%	67.70±0.27	-15.5%	0.884	0.048	–
$ART_6(\theta$=0.85)	15.1	68%	69.18±2.28	-13.6%	0.886	0.047	-14.7%
$RT_6\times2$	23	100%	56.63±0.88	-29.3%	0.869	0.059	–
$RT_6\times3$	29	100%	53.34±1.18	-33.4%	0.861	0.063	–
$ART_6\times3(\theta$=0.40)	24.7	100%	55.74±1.25	-30.4%	0.864	0.062	–
$ART_6\times3(\theta$=0.50)	21.7	100%	59.91±0.54	-25.2%	0.871	0.056	–
$ART_6\times3(\theta$=0.60)	18.8	100%	61.44±0.49	-23.3%	0.876	0.054	–

Table 13: Adaptive round-trip refinement, seeds 1–3 (3×256, GPT-2 Large). "passes": mean generator passes per sample; "refined": share of samples that received at least one round. θ is the non-canonicality (share of level 0–5 codes the re-encode changes) above which a round is run. The frontier column is the TinyStories temperature curve at the same distinct-3, or the LM1B curve at the same entropy; "–" where a sampler is more diverse than any point of the curve.

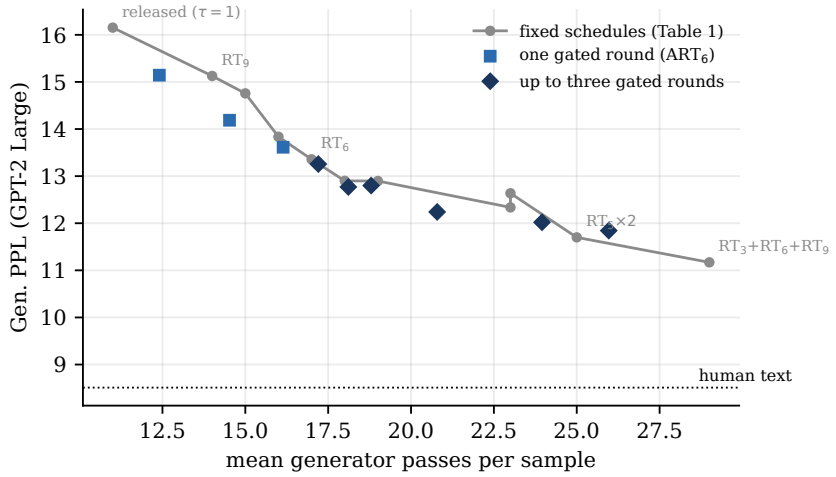


Figure 9: Quality against mean passes per sample on TinyStories. Grey: the fixed schedules of Table 1. Blue squares: one round from level 6, gated by the draft's non-canonicality (θ=0.80, 0.75, 0.70 from left to right). Dark diamonds: up to three gated rounds. Gating buys a slightly better curve than choosing a finer fixed restart level; it does not change the picture.

What the gates buy. On TinyStories a gated single round sits slightly below the fixed-schedule curve at every budget: $ART_6(0.70)$ refines 86% of stories for 16.1 passes and reaches 13.61, against 13.84 for RT_7 at 16 passes;

$\mathrm{ART}_6(0.75)$ refines 59% for 14.5 passes and reaches 14.19, against 14.76 for RT_8 at 15; and $\mathrm{ART}_6(0.80)$ refines only 23% of stories, for 1.4 extra passes, and matches RT_9 at 14 passes (15.14 vs. 15.13). The gated samplers also keep diversity closer to the released sampler (distinct-3 0.740–0.745), because most stories are left untouched. The same holds against the temperature frontier: at matched distinct-3 the gated rounds are 3–11% better than temperature, the fixed restarts RT_9–RT_7 4–8%. Early stop over three rounds behaves the same way: $\mathrm{ART}_6{\times}3(0.50)$ costs 17.2 passes and scores 13.26 against 13.36 for RT_6 at 17; $\mathrm{ART}_5{\times}3(0.50)$ costs 20.8 passes and scores 12.24, better than either fixed two-round schedule at 23 passes (12.34, 12.64) and 14.0% below the frontier against 10–12%; above 24 passes the gated samplers are slightly worse in raw perplexity than $\mathrm{RT}_5{\times}2$ and $\mathrm{RT}_5{\times}3$ but still equal or better at matched diversity, because the rounds they skip are the ones that would have narrowed the text. On LM1B large the gates are neutral: every draft is rewritten heavily, so the single-round gate at θ=0.80 refines 71% of sequences and lands on the fixed curve (−15.5% at 15.3 passes against −16.2% for RT_7 at 16), and the early-stopped three-round samplers trace the RT_6–$\mathrm{RT}_6{\times}2$–$\mathrm{RT}_6{\times}3$ line within two points. The gates are therefore a way to choose a point on the quality–compute curve per sample rather than per corpus, worth 1–2 passes or 0.3–0.6 Gen. PPL on TinyStories, and nothing on LM1B. We report them for completeness and because the selector is free; they do not alter the paper's claims.

5.11 Wall-clock cost

Measured times. Table 14 reports measured times on one RTX 4090 with the authors' batch sizes, split into generator passes and tokenizer work (encoding the decoded text, decoding and token picking). Counting passes understates the cost, because the passes a round repeats are the fine levels, which process the most positions: RT_6 takes 1.9× the time of the released sampler for 1.55× its passes on both datasets, two rounds 2.8–2.9×, and three rounds 3.8× on TinyStories. The tokenizer side (re-encoding, decoding, token picking) is about a tenth of the total, so the cost is almost entirely generator passes, and nothing in the method is a hidden bottleneck. The twins of Section 5.9 cost exactly the released sampler's time, which is why we report the hot-draft recipes as schedule plus round trip: the schedule part is free, the round-trip part is not. In absolute terms a refined 128-token story costs 10.5 ms and an LM1B sequence 65 ms, against 5.6 and 35 ms unrefined; the trade is roughly a doubling of sampling time for a 17–22% perplexity gain, which the reader can weigh against NCM's 10–31× advantage over diffusion samplers.

Sampler	NFE	ms / sample	generator (s)	tokenizer (s)	time	NFE
TinyStories: 256 stories of 128 tokens, batch 128						
released (τ=1)	11	5.6	1.21	0.20	1.00×	1.00×
RT_6	17	10.5	2.37	0.29	1.88×	1.55×
RT_6, draft τ=1.1	17	10.6	2.39	0.29	1.91×	1.55×
$\mathrm{RT}_5{\times}2$	25	16.0	3.57	0.49	2.87×	2.27×
RT_3+RT_6+RT_9	29	21.3	4.74	0.69	3.83×	2.64×
LM1B large: 128 sequences of 128 tokens, batch 32						
released (τ=1)	11	35.1	4.08	0.27	1.00×	1.00×
RT_6	17	64.7	7.83	0.43	1.84×	1.55×
$\mathrm{RT}_6{\times}2$	23	97.9	11.83	0.69	2.79×	2.09×
$\mathrm{RT}_6{\times}2$, draft τ=1.2	23	98.4	11.90	0.69	2.80×	2.09×

Table 14: Wall-clock on one RTX 4090 (PyTorch 2.5, no compilation), CUDA-synchronised, after a warm-up run. "generator" is the time inside the authors' `generate`; "tokenizer" the encoder pass on the decoded text plus decoding and token picking. The last two columns are ratios to the released sampler.

Compilation and CUDA graphs. The times above are the authors' code as released, in eager PyTorch, and a reader may ask how much of the cost of the extra passes is kernel-launch overhead that compilation would remove. Table 15 re-times the released sampler and RT_6 on the TinyStories model with the per-level generator pass compiled by `torch.compile`, once in its default mode (kernel fusion) and once in `reduce-overhead` mode (fusion plus CUDA-graph replay); sampling and the tokenizer stay eager, and each of the 11 level shapes is compiled once. At the paper's batch of 128 fusion saves 13% and CUDA graphs add nothing on top of it: the GPU is busy, and the passes are real work. At batch 8, the latency regime, launch overhead is most of the eager time and CUDA graphs remove 59% of it for both samplers, so that the refined sampler with graphs (12.0 ms per story) is faster than the released sampler as shipped (14.0 ms). The ratio between the two samplers stays at 2.1× under graphs at both batch sizes (fusion alone narrows it to 1.75× at batch 8, where the smaller released passes gain more from fusion):

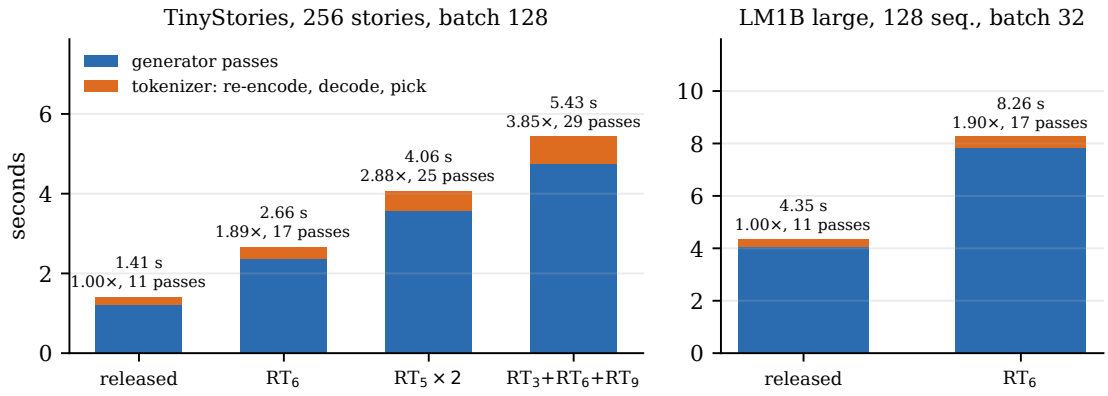


Figure 10: Where the time goes (Table 14): measured wall-clock per variant on one RTX 4090, split into generator passes and tokenizer work. The tokenizer side stays about a tenth of the total, so the cost of refinement is the extra generator passes and nothing else.

compilation lowers both bars by the same factor and leaves the trade-off of this section where it is, while being a free win for anyone deploying either sampler at small batch.

	ms / sample			vs. eager released	
Generator pass	released	RT_6	RT_6 / released	released	RT_6
batch 128					
eager (as released)	5.02	10.39	2.07×	1.00×	2.07×
compiled, fusion only	4.37	9.06	2.08×	0.87×	1.81×
compiled + CUDA graphs	4.37	9.08	2.08×	0.87×	1.81×
batch 8					
eager (as released)	14.02	29.54	2.11×	1.00×	2.11×
compiled, fusion only	10.08	17.59	1.75×	0.72×	1.25×
compiled + CUDA graphs	5.74	12.04	2.10×	0.41×	0.86×

Table 15: The same two samplers with the per-level generator pass compiled (TinyStories model, one RTX 4090, PyTorch 2.5, median of 7 timed runs after warm-up, CUDA-synchronised, decoding included). Fusion helps at both batch sizes; CUDA graphs matter only at small batch, where they make RT_6 cheaper than the eager released sampler. The ratio between the samplers is unchanged.

5.12 Does it transfer to images?

VAR. Next-scale image models face the same structural situation: VAR [30] samples a stack of multiscale residual codes and decodes it with a VQ-VAE. We applied the round trip to the released VAR-d16 ImageNet-256 model with its FID sampling settings, copying its sampler verbatim and only forcing the coarse scales: decode the draft to an 8-bit image, re-encode it with VAR's own VQ-VAE, keep scales $< k$ and regenerate the rest (Table 16; 10 images per class, FID-10k against the ADM reference images with Inception features from `pytorch-fid`). Forcing every scale to the draft's own codes reproduces all 2,000 test drafts exactly. The phenomenon carries over: the image round trip changes 29–92% of the codes at the first nine scales, the same order as on text, so VAR too samples stacks its tokenizer would not produce. The benefit does not. The best restart moves FID from 9.13 to 8.92, which looks like a gain until the control is read: resampling without re-encoding, which only redraws from the released distribution, already moves FID to 9.22, so differences of 0.1–0.2 are what this 10k-sample measurement produces by chance, and the 8.92 is within that band. Restarting from coarser scales is worse (9.30, 9.35). Two differences from text may explain why canonical coarse codes do not help here: unlike text, the pixel round trip is lossy, so the re-encoded coarse scales describe a slightly different image rather than the same one; and VAR conditions every scale on all earlier scales rather than on the latest reconstruction only, so a corrected coarse scale is only one of many inputs to the fine ones. We did not run the 50k-sample confirmation.

Sampler (VAR-d16, ImageNet 256)	FID-10k ↓	codes changed by the round trip
VAR sampler (cfg 1.5, top-k 900, top-p 0.96)	9.13	–
Resample_7 (no re-encode)	9.22	–
LatentRQ_7	9.26	26–92%
RT_8	8.92	29–92%
RT_7	9.13	29–92%
RT_6	9.30	29–92%
RT_5	9.35	29–92%
$\text{RT}_7 \times 2$	9.25	29–92%

Table 16: Round-trip refinement on an image next-scale model. The round trip goes through an 8-bit image. Code changes are per scale, over the first nine of ten scales.

6 What did not work

Setup. Before finding round-trip refinement we tried six training-side ideas on the TinyStories checkpoint, two of which the NCM paper names as future work, and afterwards two at the paper's batch size. All fine-tunes continue the released generator with AdamW (0.9, 0.95), weight decay 0.1 and gradient clipping 1, as in the paper, but with batch 64–128 instead of 2,048 for budget reasons, except the last two, 200-step pilots at batch 2,048 (self-correction targets on two H100s; own-error corruption re-run on two A100s). Table 17 summarises them; numbers are GPT-2 Large Gen. PPL at full depth unless stated.

Idea	Outcome
Early exit (adaptive depth)	No plateau to exit into: the last pass alone takes GPT-2 small Gen. PPL from 37 to 20.5; the best per-sample stopping rule saves 0.7 passes for +3%.
Level skipping at inference	One skipped middle level +6–11%, three +34%, five 4.9–7×, although the tokenizer still reconstructs 98–99% of tokens from every other level.
Level-dropout fine-tuning	With the paper's conditioning corruption, no gain at 8–10 passes; wins only at 6 passes (54.6 vs. 79.7), still 3.4× worse than full depth. Without corruption, any fine-tune collapses to 36–40 (exposure bias).
Joint tokenizer + generator	Tie: 16.83 (with a coupling loss) vs. 16.85 for the generator-only control; reconstruction stays 99.98%.
Own-error corruption	Replacing uniform corruption by the model's own sampled guesses (which disagree with the tokenizer's code at 47% of positions) hurts, dose-dependently: 21.8 (half) and 40.2 (all) vs. 16.85 at batch 128. Re-run at batch 2,048 (200 steps, seeds 4–6) the damage is the same size, 20.9 and 38.2 vs. 16.8, but it is a shift in diversity, distinct-3 0.79 and 0.85 against 0.75: matched for diversity against the released model's own per-level temperature schedules on the same seeds, both corrupted models tie within ±3% (half at τ=0.9: 16.9 at 0.763 vs. 17.1; all at τ=0.8: 18.1 at 0.775 vs. 18.0), and the round trip on top of them ties the schedule-plus-round-trip twins (15.4 vs. 15.9 at 0.77). Training on its own guesses makes the sampler hotter, not better.
Distilling refinement	Training on 50% refined stories matches the released 16.15 at 11 passes, but at the diversity of τ=0.8 (Table 18).
Self-correction targets at batch 2,048	At the paper's batch size the control fine-tune no longer drifts (200 steps: 16.94 at distinct-3 0.753, 4% *better* than the released temperature curve at that diversity, seeds 4–6). Re-quantizing the targets after corrupting the inputs, as in Infinity's bitwise self-correction [10], ties with the control under uniform corruption (17.13) and, under nearest-code corruption, yields a far more diverse sampler (distinct-3 0.806) that is still 6% worse than the released model's best per-level temperature schedule at that diversity (23.4 vs. 18.1).

Table 17: Training-side ideas that did not beat the released sampler.

Why distillation fails. The distillation result deserves a comment, because without diversity metrics it looks like a near-win. The distilled model scores 16.15, exactly the released sampler's number, at the released 11 passes; against its own control (16.78) it is 3.8% better on every seed, and fine-tuning drift (the control is 3.9% worse than the released model) seems to be the only obstacle. Corpus diversity tells a different story. The distilled model's distinct-3 is 0.719, the value of the released sampler at τ=0.8, and at that diversity the released sampler scores

Generator (11 passes, τ=1)	batch	Gen. PPL ↓	dist-3 ↑	xs-4 ↓
released (no fine-tune)	–	16.15	0.749	0.162
control fine-tune	128	16.78	0.755	0.158
control, soup	128	16.43	0.750	0.162
distilled	128	16.15	0.719	0.188
distilled, soup	128	16.00	0.734	0.176
released, τ=0.8	–	13.34	0.719	0.191

Table 18: Distilling round-trip refinement into the generator: 1,500 fine-tuning steps on half real stories and half RT_5×2-refined stories, against a control on real stories only; "soup" averages the fine-tune with the released weights [33]. The distilled model ties the released Gen. PPL only because it is less diverse: the released sampler at the same distinct-3 (τ=0.8) scores 13.34.

13.34, not 16.15. Sequence-level distillation [11] thus transferred the refined sampler's lower diversity but not its fluency at that diversity: at matched distinct-3 the distilled model is about 21% worse than simply lowering the released model's temperature. In other words, the student learned to be narrower, not better. A larger batch would reduce the drift but not this gap. The common thread of Table 17 is that, at this scale, inference-time changes to the released model were far cheaper and more effective than any fine-tune we could afford.

7 Related work

Coarse-to-fine and residual-code generation. NCM [13] brings next-scale prediction [12, 30] and residual quantization [3, 14] to text. Infinity's bitwise self-correction [10] re-quantizes *training* targets after flipping input bits so that later scales learn to fix earlier errors; our method instead re-quantizes *at inference*, needs no retraining, and goes through the decoded tokens.

Re-quantize and regenerate. The closest prior method is scale-travel in DiverseVAR [21]: re-quantize the accumulated image latent, rewind to a coarser scale and regenerate, used there to restore quality after diversity-boosting perturbations; no base-sampler result is reported. Round-trip refinement differs in going through the discrete output (decode, then re-encode) rather than re-quantizing the latent, in targeting text, and in being evaluated as a base sampler with frontier controls; LatentRQ_k (Section 5.4) is the direct comparison; on NCM it recovers about half of the matched-diversity gain of the text round trip. SDEdit [19] similarly re-noises an output and denoises it again with a diffusion model, and Restart sampling [34] alternates forward noising and backward sampling to correct accumulated errors, the closest mechanism in continuous diffusion. TTS-VAR [6] spends test-time compute on scale-wise image generation through selection.

Iterative refinement and self-correction. Mask-Predict [9], MaskGIT [4], Token-Critic [16] and ReMDM remasking [31] revise parallel predictions by re-masking and re-predicting tokens; Self-Refine [18] has large LMs critique and rewrite their own outputs. In NCM, codes within a level are conditionally independent given the coarser levels, so re-masking within a level does not apply; the round trip changes the *conditioning* instead.

Exposure bias. Training on model outputs [1, 29] and student-forcing for scale-wise image models [36] address the train–test gap in training; our own-error corruption experiment is in this family and did not help in fine-tuning at either batch size: at the paper's batch size it reproduces the released model's temperature frontier rather than moving it. Joint or post-training of tokenizers [15, 25, 32] helps in vision; for NCM text at our scale it tied.

Evaluation. Gen. PPL can be lowered by reducing diversity or by low-precision sampling [8, 24, 35]; we therefore report temperature frontiers and corpus diversity alongside it.

8 Limitations

Our evaluation relies on Gen. PPL under GPT-2 Large (checked against Qwen2.5-7B), simple diversity statistics, MAUVE and a single 7B LLM judge with a strong position bias that we average out; we did not run a human evaluation, and at equal Gen. PPL the judge does not distinguish refinement from a lower temperature. All models are around 100M parameters with 128-token sequences; the authors list an OpenWebText checkpoint, which would be the only model of long general text, but its weights were not on the hub at the time of writing, so we could not

test it. Refinement costs 1.3–2.6× the generator passes, 1.5–3.3× the generator FLOPs and 1.9–3.8× the measured wall-clock, a ratio that compilation and CUDA graphs do not change (Table 15). It fails when the tokenizer round trip is lossy (WMT14), and it does not transfer to the image model we tried (VAR-d16, FID-10k only). Our reproduction of the released TinyStories sampler is 5% above the paper's number, and all relative claims are against our reproduction. The train–test mismatch is our explanation of the gain, supported by the canonicality correlations and the ablations but not tested causally; the direct tests, a generator trained on its own sampled stacks, are the self-correction and own-error pilots of Section 6, which both tied with the released model at matched diversity. Restart levels and hot-draft temperatures were chosen on the same three seeds we report, from small grids. Most fine-tuning baselines were limited to batch 64–128, and a 200-step pilot at batch 2,048 showed that the control then no longer drifts, so the earlier training-side negative results may partly reflect the small batches, with the exception of the diversity argument against distillation and the two batch-2,048 pilots themselves; the own-error result in particular is the same at both batch sizes once diversity is matched.

9 Conclusion

NCM's coarse-to-fine sampler produces code stacks that its own tokenizer would never produce for the same text. Passing the draft through the tokenizer and regenerating the fine levels repairs this, with no training and a few lines of code, and lowers generative perplexity by 17–31% on TinyStories and 22–29% on LM1B while keeping diversity, 10–14% beyond what temperature can buy and 15–25% when paired with a per-level temperature schedule. The ablations locate the gain in the re-encoding itself: resampling without it does nothing, and re-quantizing the latent instead of the text gives about half. The results hold on held-out seeds and do not shrink with model size; exact temperature twins and real human text show that the gain is neither a temperature schedule nor a sharpener; an LLM judge prefers refined stories; MAUVE moves closer to human text; and on Sudoku the gain is limited to the hardest puzzles, while verifier-guided search is better served by plain resampling. The method needs a lossless tokenizer round trip, and it trades extra passes for quality, so it complements rather than replaces NCM's main appeal of speed. Training-time alternatives that we could afford, including distilling the refined sampler, did not help.

Acknowledgements. We thank the NCM authors for releasing checkpoints, data and evaluation code, without which none of this would have been possible. This work was carried out independently of any King's College London course.

References

[1] Samy Bengio, Oriol Vinyals, Navdeep Jaitly, and Noam Shazeer. Scheduled sampling for sequence prediction with recurrent neural networks. *arXiv preprint arXiv:1506.03099*, 2015.

[2] Ondřej Bojar, Christian Buck, Christian Federmann, Barry Haddow, Philipp Koehn, Johannes Leveling, Christof Monz, Pavel Pecina, Matt Post, Herve Saint-Amand, Radu Soricut, Lucia Specia, and Aleš Tamchyna. Findings of the 2014 workshop on statistical machine translation. In *Proceedings of the Ninth Workshop on Statistical Machine Translation*, 2014.

[3] Zalán Borsos, Matt Sharifi, Damien Vincent, Eugene Kharitonov, Neil Zeghidour, and Marco Tagliasacchi. SoundStorm: Efficient parallel audio generation. *arXiv preprint arXiv:2305.09636*, 2023.

[4] Huiwen Chang, Han Zhang, Lu Jiang, Ce Liu, and William T. Freeman. MaskGIT: Masked generative image transformer. *arXiv preprint arXiv:2202.04200*, 2022.

[5] Ciprian Chelba, Tomas Mikolov, Mike Schuster, Qi Ge, Thorsten Brants, Phillipp Koehn, and Tony Robinson. One billion word benchmark for measuring progress in statistical language modeling. *arXiv preprint arXiv:1312.3005*, 2013.

[6] Zhekai Chen, Ruihang Chu, Yukang Chen, Shiwei Zhang, Yujie Wei, Yingya Zhang, et al. TTS-VAR: A test-time scaling framework for visual auto-regressive generation. *arXiv preprint arXiv:2507.18537*, 2025.

[7] Ronen Eldan and Yuanzhi Li. TinyStories: How small can language models be and still speak coherent English? *arXiv preprint arXiv:2305.07759*, 2023.

[8] Antonio Franca and Alexander Tong. Hacking generative perplexity: Why unconditional text evaluation needs distributional metrics. *arXiv preprint arXiv:2606.08417*, 2026.

[9] Marjan Ghazvininejad, Omer Levy, Yinhan Liu, and Luke Zettlemoyer. Mask-predict: Parallel decoding of conditional masked language models. *arXiv preprint arXiv:1904.09324*, 2019.

[10] Jian Han, Jinlai Liu, Yi Jiang, Bin Yan, Yuqi Zhang, Zehuan Yuan, Bingyue Peng, and Xiaobing Liu. Infinity: Scaling bitwise autoregressive modeling for high-resolution image synthesis. *arXiv preprint arXiv:2412.04431*, 2024.

[11] Yoon Kim and Alexander M. Rush. Sequence-level knowledge distillation. *arXiv preprint arXiv:1606.07947*, 2016.

[12] Hermann Kumbong, Xian Liu, Tsung-Yi Lin, Ming-Yu Liu, Xihui Liu, Ziwei Liu, Daniel Y. Fu, Christopher Ré, and David W. Romero. HMAR: Efficient hierarchical masked auto-regressive image generation. *arXiv preprint arXiv:2506.04421*, 2025.

[13] Hermann Kumbong, Ishan S. Khare, Iddah Mlauzi, Andrew Woen, Charles Ding, E. Kelly Buchanan, Simran Arora, Azalia Mirhoseini, and Christopher Ré. Next-concept modeling: Fast, parallel coarse-to-fine text generation. *arXiv preprint arXiv:2610.14162*, 2026.

[14] Doyup Lee, Chiheon Kim, Saehoon Kim, Minsu Cho, and Wook-Shin Han. Autoregressive image generation using residual quantization. *arXiv preprint arXiv:2203.01941*, 2022.

[15] Xingjian Leng, Jaskirat Singh, Yunzhong Hou, Zhenchang Xing, Saining Xie, and Liang Zheng. REPA-E: Unlocking VAE for end-to-end tuning with latent diffusion transformers. *arXiv preprint arXiv:2504.10483*, 2025.

[16] José Lezama, Huiwen Chang, Lu Jiang, and Irfan Essa. Improved masked image generation with token-critic. *arXiv preprint arXiv:2209.04439*, 2022.

[17] Chin-Yew Lin. ROUGE: A package for automatic evaluation of summaries. In *Text Summarization Branches Out*, 2004.

[18] Aman Madaan, Niket Tandon, Prakhar Gupta, Skyler Hallinan, Luyu Gao, Sarah Wiegreffe, Uri Alon, Nouha Dziri, et al. Self-refine: Iterative refinement with self-feedback. *arXiv preprint arXiv:2303.17651*, 2023.

[19] Chenlin Meng, Yutong He, Yang Song, Jiaming Song, Jiajun Wu, Jun-Yan Zhu, and Stefano Ermon. SDEdit: Guided image synthesis and editing with stochastic differential equations. *arXiv preprint arXiv:2108.01073*, 2021.

[20] Shashi Narayan, Shay B. Cohen, and Mirella Lapata. Don't give me the details, just the summary! topic-aware convolutional neural networks for extreme summarization. *arXiv preprint arXiv:1808.08745*, 2018.

[21] Mingue Park, Prin Phunyaphibarn, Phillip Y. Lee, and Minhyuk Sung. DiverseVAR: Balancing diversity and quality of next-scale visual autoregressive models. *arXiv preprint arXiv:2511.21415*, 2025.

[22] Krishna Pillutla, Swabha Swayamdipta, Rowan Zellers, John Thickstun, Sean Welleck, Yejin Choi, and Zaid Harchaoui. MAUVE: Measuring the gap between neural text and human text using divergence frontiers. *arXiv preprint arXiv:2102.01454*, 2021.

[23] Matt Post. A call for clarity in reporting BLEU scores. *arXiv preprint arXiv:1804.08771*, 2018.

[24] Patrick Pynadath, Jiaxin Shi, and Ruqi Zhang. Generative frontiers: Why evaluation matters for diffusion language models. *arXiv preprint arXiv:2604.02718*, 2026.

[25] Kai Qiu, Xiang Li, Hao Chen, Jason Kuen, Xiaohao Xu, Jiuxiang Gu, Yinyi Luo, Bhiksha Raj, et al. Image tokenizer needs post-training. *arXiv preprint arXiv:2509.12474*, 2025.

[26] Qwen Team. Qwen2.5 technical report. *arXiv preprint arXiv:2412.15115*, 2024.

[27] Alec Radford, Jeffrey Wu, Rewon Child, David Luan, Dario Amodei, and Ilya Sutskever. Language models are unsupervised multitask learners. Technical report, OpenAI, 2019.

[28] Subham Sekhar Sahoo, Marianne Arriola, Yair Schiff, Aaron Gokaslan, Edgar Marroquin, Justin T. Chiu, Alexander Rush, and Volodymyr Kuleshov. Simple and effective masked diffusion language models. *arXiv preprint arXiv:2406.07524*, 2024.

[29] Nikolay Savinov, Junyoung Chung, Mikolaj Binkowski, Erich Elsen, and Aaron van den Oord. Step-unrolled denoising autoencoders for text generation. *arXiv preprint arXiv:2112.06749*, 2021.

[30] Keyu Tian, Yi Jiang, Zehuan Yuan, Bingyue Peng, and Liwei Wang. Visual autoregressive modeling: Scalable image generation via next-scale prediction. *arXiv preprint arXiv:2404.02905*, 2024.

[31] Guanghan Wang, Yair Schiff, Subham Sekhar Sahoo, and Volodymyr Kuleshov. Remasking discrete diffusion models with inference-time scaling. *arXiv preprint arXiv:2503.00307*, 2025.

[32] Hanyu Wang, Saksham Suri, Yixuan Ren, Hao Chen, and Abhinav Shrivastava. LARP: Tokenizing videos with a learned autoregressive generative prior. *arXiv preprint arXiv:2410.21264*, 2024.

[33] Mitchell Wortsman, Gabriel Ilharco, Jong Wook Kim, Mike Li, Simon Kornblith, Rebecca Roelofs, Raphael Gontijo-Lopes, Hannaneh Hajishirzi, et al. Robust fine-tuning of zero-shot models. *arXiv preprint arXiv:2109.01903*, 2021.

[34] Yilun Xu, Mingyang Deng, Xiang Cheng, Yonglong Tian, Ziming Liu, and Tommi Jaakkola. Restart sampling for improving generative processes. *arXiv preprint arXiv:2306.14878*, 2023.

[35] Kaiwen Zheng, Yongxin Chen, Hanzi Mao, Ming-Yu Liu, Jun Zhu, and Qinsheng Zhang. Masked diffusion models are secretly time-agnostic masked models and exploit inaccurate categorical sampling. *arXiv preprint arXiv:2409.02908*, 2024.

[36] Gengze Zhou, Chongjian Ge, Hao Tan, Feng Liu, and Yicong Hong. Rethinking training dynamics in scale-wise autoregressive generation. *arXiv preprint arXiv:2512.06421*, 2025.

A Implementation

Refinement reuses the released `NextConceptTransformer.generate`. The only change is a wrapper around its per-level sampling function:

```
_orig = net._sample_level
def _sample_with_force(logits, temperature, top_k, top_p):
    level = state["calls"] + 1 - state["offset"]   # level-0 queries counted by offset
    state["calls"] += 1
    if state["codes"] is not None and 1 <= level < state["upto"]:
        return state["codes"][level]               # re-encoded codes for levels < k
    return _orig(logits, temperature, top_k, top_p)
net._sample_level = _sample_with_force

draft = generate(l0)                                # authors' sampler, unchanged
codes = split_levels(tokenizer.vqvae.encode(draft)) # re-encode the decoded tokens
refined = generate(codes[0], forced=codes, upto=k)  # regenerate levels k..K-1
```

All scripts, run logs and per-seed results are in the accompanying repository.

B Samples

TinyStories, seed 1, sample 4 (same draft for all rows; differences from the draft in bold).

Draft (released sampler). One day, a little cat named Amy went to a big room to play. In the room, there were many toys **fun There** were games, and Amy felt happy. She looked and saw a cat. The cat was happy too. The cat tried to catch Amy, but she did not know how. The cat tried to jump high **to to** reach the cat. Amy felt sad and missed the sad cat. …

RT_6**.** One day, a little cat named Amy went to a big room to play. In the room, there were many toys**. There** were games, and Amy felt happy. She looked and saw a cat. The cat was happy too. The cat tried to catch Amy, but she did not know how. The cat tried to jump **up high to** reach the cat. Amy felt sad and missed the sad cat. …

RT_3+RT_6+RT_9**.** One day, a little girl named Amy went to the gym to play. In the gym, there were many kids laughing and playing games. Amy was very happy sound. She looked around and saw a furry cat. The cat was very sad. Amy wanted to help the cat, but she did not know how. She tried to jump high, but she fell down too. …

A restart at level 6 keeps the story and repairs local errors; a restart at level 3 re-plans the content ("cat named Amy" becomes "girl named Amy") and, as Table 1 shows, trades a little diversity for the largest fluency gain.

Use of AI tools

Code and experiments were developed with the assistance of an AI coding assistant (Claude, Anthropic). The author designed the study, chose what to run, checked every number against the run logs and is responsible for all content.